



Amazon Virtual Private Cloud (VPC)

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Virtual Private Cloud (VPC)



Provision a **logically isolated section of the AWS Cloud** where you can launch AWS resources in a **virtual network** that you define

Introduction to Virtual Private Cloud

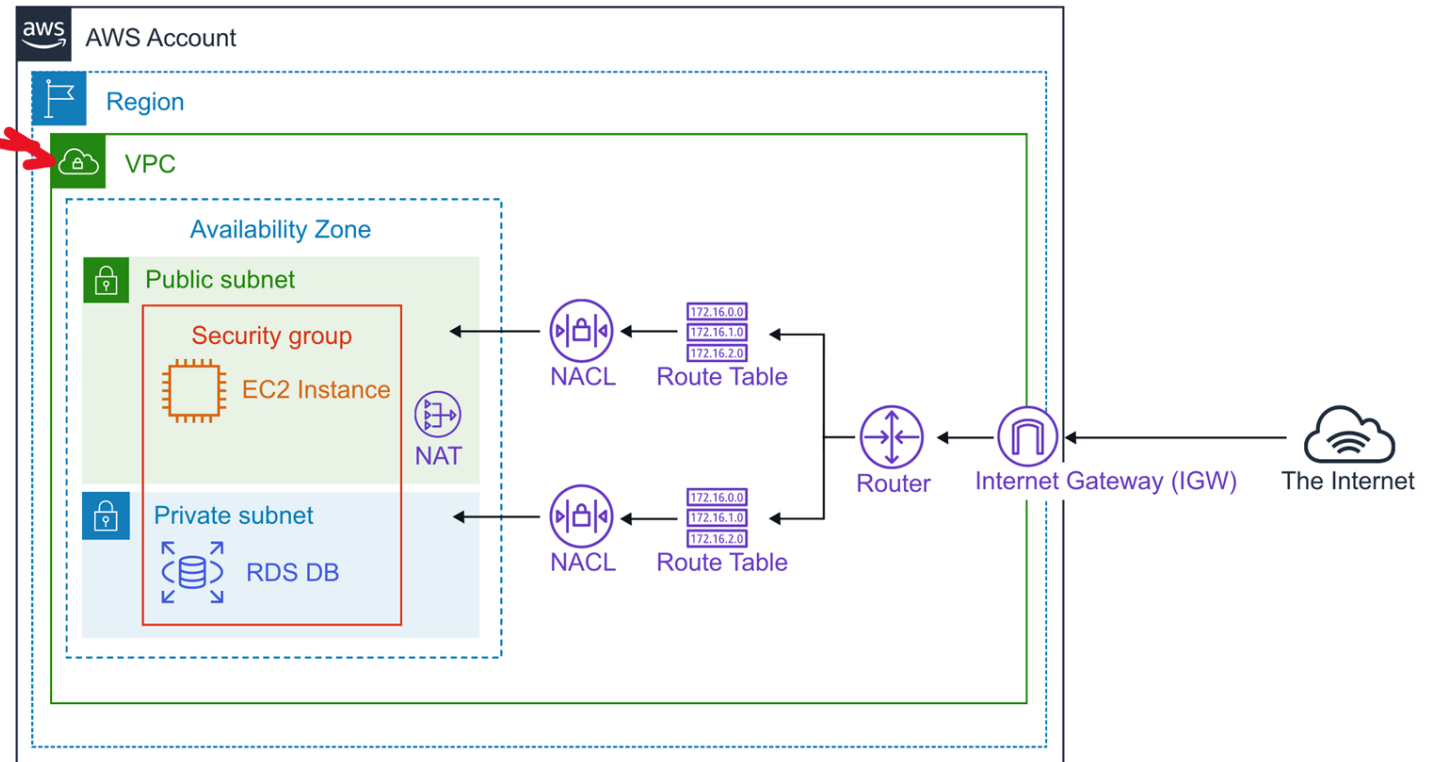


AWS Virtual Private Cloud (VPC) is a **logically isolated virtual network**.
AWS VPC resembles a traditional network you'd operate in your own data center.

A **VPC** provides many different networking components

Virtual Machines eg. EC2 is the most common reason for using a VPC in AWS.

Virtual Network cards eg. Elastic Network Interfaces (ENIs) is used to within a VPC to attach to different compute types eg. EC2, Lambda, ECS



AWS VPC is tightly coupled with **AWS EC2**, and *all VPC CLI commands are under aws ec2

Core Components of VPC

A VPC is composed of many different networking components:



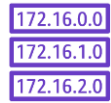
Internet Gateway (IGW)

A gateway that connects your VPC to the internet



Virtual Private Gateway (VPN Gateway)

A gateway that connects your VPC to a private external network



Route Tables

determines where to route traffic within a VPC



NAT Gateway

Allows private instances (eg. Virtual Machines) to connect to services outside the VPC



Network Access Control Lists (NACLs)

Acts as a **stateless** virtual firewall for compute within a VPC
Operates at the subnet level with allow and deny rules

Security Groups (SG)

Acts as a **stateful** virtual firewall for compute within a VPC
Operates at the instance level with Allow rules



Public Subnets

Subnets allow instances to have public IP addresses



Private Subnets

Subnets that disallow instances to have public IP addresses



VPC Endpoints

Privately connect to AWS support services



VPC Peering

Connecting VPCs to other VPCs

Technically, NACLs and SGs are EC2 networking components.

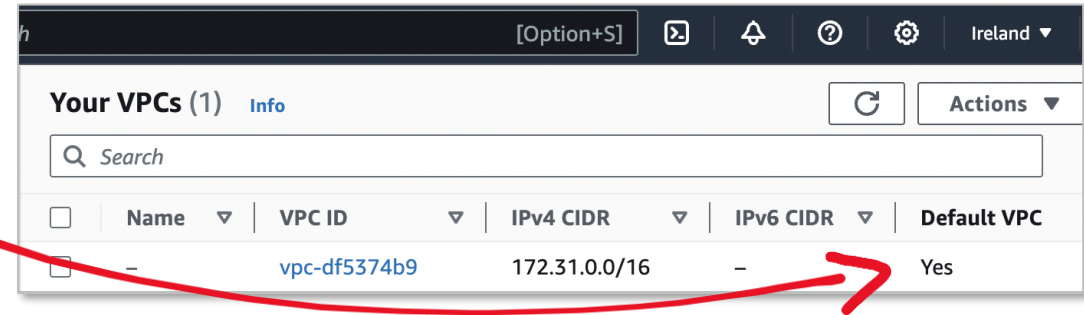
Key Features

- VPCs are **Region Specific**. They do not span regions
 - You can use VPC Peering to connect VPCs across regions
- You can create up to **5 VPC** per region (adjustable).
- Every region comes with a default VPC
- You can have **200 subnets** per VPC
- Up to **5 IPv4 CIDR Blocks** per VPC (adjustable to 50)
- Up to **5 IPv6 CIDR Blocks** per VPC (adjustable to 50)
- **Most Components Cost nothing:**
 - VPCs, Route Tables, NACLs, Internet Gateways, Security Groups and Subnets, VPC Peering
- **Some things cost money:** eg. NAT Gateway,
 - VPC Endpoints, VPN Gateway, Customer Gateway
 - IPV4 Addresses, Elastic IPs
- **DNS hostnames** (should your instance have domain name addresses)

Default VPC



AWS has a default VPC in every region
So you can **immediately** deploy instances.



	Name	VPC ID	IPv4 CIDR	IPv6 CIDR	Default VPC
☐	-	vpc-df5374b9	172.31.0.0/16	-	Yes

A default VPC is configured by default with:

- An IPv4 CIDR block at the address **172.31.0.0/16**. **65,536** IPv4 addresses
- A subnet (size of /20) for each possible Availability Zone (AZ) **4,096** IPv4 addresses
- An Internet Gateway (IGW)
- A default Security Group (SG)
- Create a default Network access control list (NACL)
- **Default DHCP** options set
- A Route Table with a route out to the internet via IGW



You CAN delete a default VPC



AWS recommends the use of the default VPC. *Other cloud service providers may recommend against using the default.

Create a Default VPC

If you delete the Default VPC (by accident or intentional) and you want to recreate a default VPC. You can run the follow AWS CLI command:

```
aws ec2 create-default-vpc --region ca-central-1
```



```
{
  "Vpc": {
    "VpcId": "vpc-3f139646",
    "InstanceTenancy": "default",
    "Tags": [],
    "Ipv6CidrBlockAssociationSet": [],
    "State": "pending",
    "DhcpOptionsId": "dopt-61079b07",
    "CidrBlock": "172.31.0.0/16",
    "IsDefault": true
  }
}
```

- You cannot restore a previous default VPC that you deleted
- You cannot mark an existing nondefault VPC as a default VPC.
- If you already have a default VPC in the Region, you cannot create another one.

Deleting a VPC

To Delete a VPC you need to **delete multiple VPC resource before you can delete:**

- Security groups (SGs) and Network ACLs (NACLs)
- Subnets
- Route tables (RTs)
- Gateway endpoints
- Internet gateways (IGWs)
- Egress-only internet gateways (EO-IGWs)



```
aws ec2 delete-security-group --group-id sg-id
aws ec2 delete-network-acl --network-acl-id acl-id
aws ec2 delete-subnet --subnet-id subnet-id
aws ec2 delete-route-table --route-table-id rtb-id
aws ec2 detach-internet-gateway --internet-gateway-id igw-id --vpc-id vpc-id
aws ec2 delete-internet-gateway --internet-gateway-id igw-id
aws ec2 delete-egress-only-internet-gateway --egress-only-internet-gateway-id eigw-id

aws ec2 delete-vpc --vpc-id vpc-id
```



When you delete a VPC in the AWS Manage Console it will **automatically** attempt to delete the resources for you

Default Route / Catch-All-Route

The **default route** or **catch-all-route** represents all possible IP addresses
Think of this route as **giving access from anywhere or to the internet without restriction**

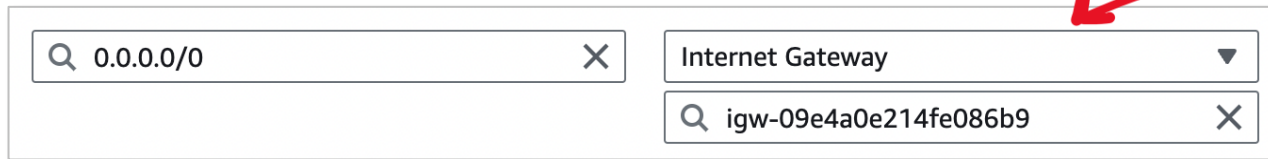
IPV4 default route

0.0.0.0/0

IPV6 default route

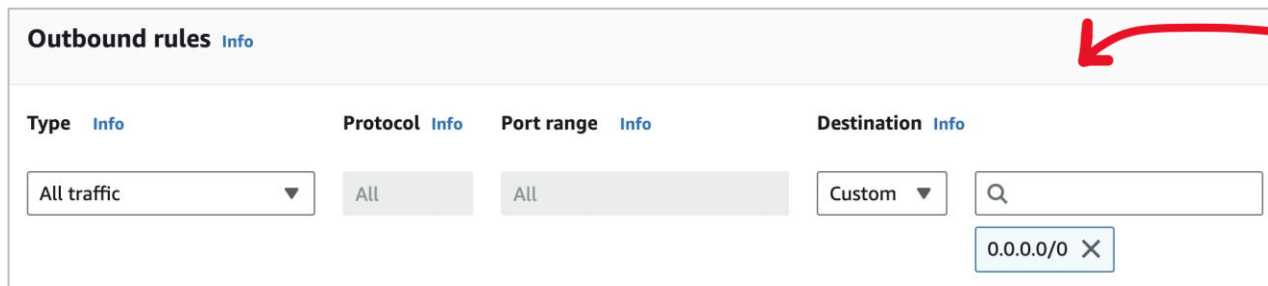
::/0

:: is shorthand for 0000:0000:0000:0000:0000:0000:0000:0000



A screenshot of the AWS Route Table configuration interface. It shows a search bar with '0.0.0.0/0' and a dropdown menu with 'Internet Gateway' selected. Below the dropdown, a search bar shows 'igw-09e4a0e214fe086b9'.

When we specify **0.0.0.0/0** in our Route Table for IGW, we are allowing internet access



A screenshot of the AWS Security Group Outbound Rules configuration interface. It shows a table with columns: Type, Protocol, Port range, and Destination. The 'Type' column has a dropdown menu with 'All traffic' selected. The 'Destination' column has a dropdown menu with 'Custom' selected and a search bar with '0.0.0.0/0' entered.

When we specify **0.0.0.0/0** in our Security Group's Inbound Rules, we are allowing all traffic from the internet to access our public resources

Network Access Control List (NACLs)

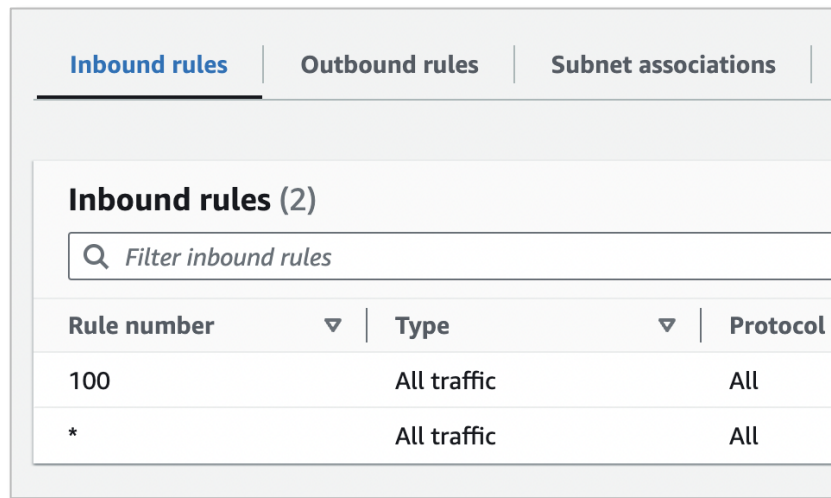
An (optional) layer of security that acts
As a **firewall** for controlling traffic **in and out of subnet(s)**

Introduction to NACLs



Network Access Controls (NACLs) acts as a **stateless virtual firewall** at the subnet level

- NACLs have both ALLOW and DENY rules.
- A default NACL is created with every VPC



Inbound rules			Outbound rules	Subnet associations
Inbound rules (2)				
Filter inbound rules				
Rule number	Type	Protocol		
100	All traffic	All		
*	All traffic	All		

Each NACL contains two different sets of rules:

- Inbound rules (ingress traffic, entering)
- Outbound rules (egress traffic, leaving)

Inbound rules

Outbound rules

Subnet associations

Subnet associations (3)

Filter subnet associations

Name	Subnet ID	Associated w...	Avai
-	subnet-9d0f9efb	acl-d81bc2a1	eu-v
-	subnet-4a4a2a02	acl-d81bc2a1	eu-v
-	subnet-0bb36151	acl-d81bc2a1	eu-v

- Subnets are associated with NACLs.
- A subnet can only belong to a single NACL.

The key difference of NACL's vs Security Groups is that NACLs have both allow and deny rule. With NACL's you **could block a single IP address**. You can't do this with SGs

Introduction to NACLs

Inbound rule

Rule number [Info](#)

Type [Info](#)

Protocol [Info](#)

Port range [Info](#)

Source [Info](#)

Allow/Deny [Info](#)

Rule # determines the **order of evaluation** from lowest to highest. The highest rule # can be 32766 and its recommended to work in 10 or 100 increments.

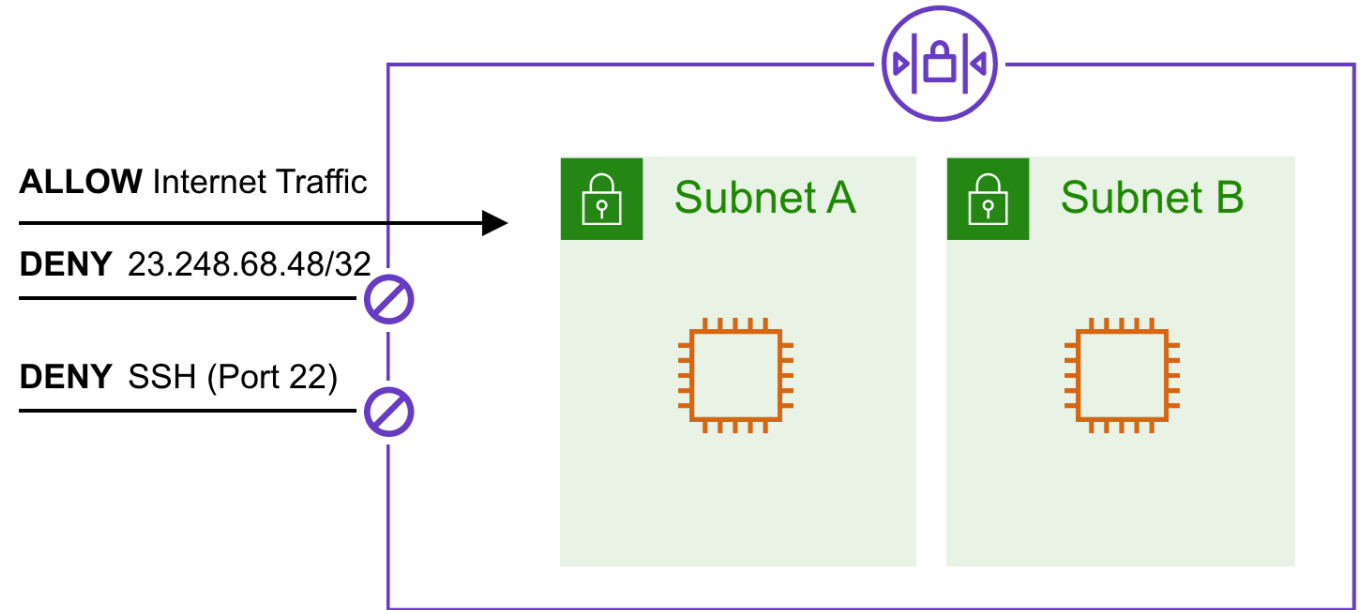
You can set a type of traffic or set up a custom **Protocol** and **Port range**.

You can either allow or deny traffic for this rule.

NACLs - Use Case

We determine there is a malicious actor at a specific IP address trying to access our instances, so we block their IP

We never need to SSH into instances, so we add a DENY for these subnets. This is just an additional measure in case our Security Groups SSH port was left open.



Creating a NACLs

We create the NACL



```
aws ec2 create-network-acl \  
--vpc-id vpc-1234abcd5678efgh
```

We add a a NACL entry (rule)



```
aws ec2 create-network-acl-entry \  
--network-acl-id acl-1a2b3c4d5e6f7890a \  
--ingress \  
--rule-number 100 \  
--protocol tcp \  
--port-range From=80,To=80 \  
--cidr-block 0.0.0.0/0 --rule-action allow
```

We associate the NACL to a subnet



```
aws ec2 replace-network-acl-association \  
--association-id aclassoc-1b2c3d4e5f67890ab \  
--network-acl-id acl-1a2b3c4d5e6f7890a
```



NACLs CheatSheet

- Network Access Control List is commonly known as NACL
- VPCs are automatically given a default NACL which allows **all** outbound and inbound traffic.
- Each subnet within a VPC must be associated with a NACL
- Subnets can only be associated with 1 NACL at a time. Associating a subnet with a new NACL will remove the previous association.
- If a NACL is not explicitly associated with a subnet, the subnet will automatically be associated with the default NACL.
- NACL has inbound and outbound rules (just like Security Groups).
- Rule can either **allow** or **deny** traffic. (unlike Security Groups which can only allow)
- NACLs are STATELESS (incoming rule will not be applied to the outgoing)
- When you create a NACL, it will deny all traffic by default
- NACLs contain a numbered list of rules that gets evaluated in order from lowest to highest.
- If you needed to block a single IP address, you could via NACLs (Security Groups cannot deny)

sg-exampro

Security Groups



A virtual **firewall that controls the traffic to and from EC2 Instances**

Introduction to Security Groups (SGs)

Security Groups (SG) act as as stateful **virtual firewall** at the **instance level**

Security groups [exampro-elb-asg-WebServerSecurityGroup-192Z5KV62TPYW](#). [view inbound rules](#). [view outbound rules](#)

Security Groups are associated with EC2 instances

Each SG contains two different sets of rules:

- Inbound rules (ingress traffic, entering)
- Outbound rules (egress traffic, leaving)

Details

Inbound rules

Outbound rules

Tags

Outbound rules (1)

Manage tags

Edit outbound rules

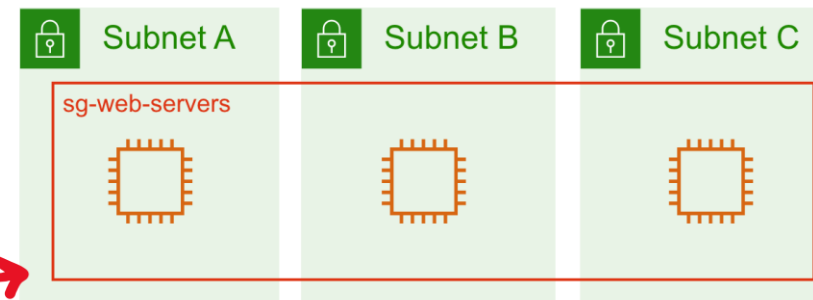
Q Search

<1>

	Name ▾	Security group r... ▾	IP versi... ▾	Type ▾	Protocol ▾	Port ra... ▾	Destination ▾
	-	sgr-0ec9a8fadf2f4...	IPv4	All traffic	All	All	0.0.0.0/0

A Security Group (Sg) can contain multiple instances in different subnets.

- SGs are not bound by subnets, but are bound by VPC



Security Group Rules

Outbound rule 1 Delete

Security group rule ID
sgr-0ec9a8fadf2f44502

Type [Info](#)
All traffic ▼

Protocol [Info](#)
All

Port range [Info](#)
All

Destination type [Info](#)
Custom ▼

Destination [Info](#)

Description - optional [Info](#)

You can choose a preset Traffic Type eg. HTTP/S, Postgres

You can choose a custom Protocol (UDP/TCP) and Port Range

Destination type can be a:

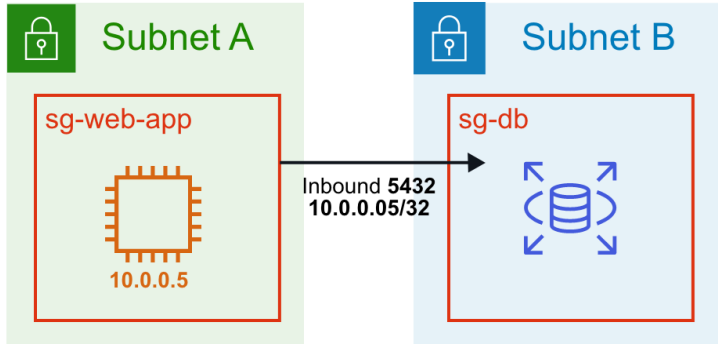
- IPv4 CIDR Block
- IPv6 CIDR Block
- Another Security Group
- Managed Prefix List

There are **only Allow Rules**

There are no 'Deny' rules.

All traffic is blocked by default

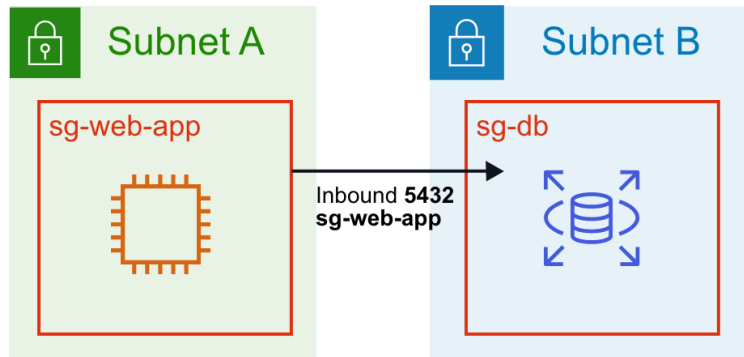
Security Groups - Use Case



Allow IP Addresses

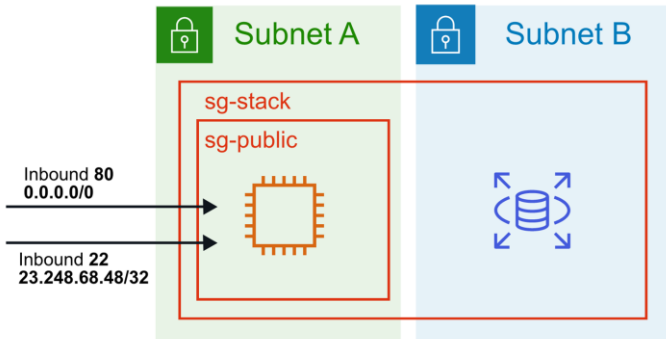
You can specify the source to be an IPv4 or Ipv6 range or a specific IP
A specific IP

- A specific IP address for IPv4 is /32
- A specific IP address for IPv6 is /128



Allow to Another Security Groups

You can specify the source to be another security group



Nested Security Groups

An instance can **belong to multiple Security Groups**, and rules are **permissive** (instead of restrictive). If you have one security group with no Allow and add an Allow to another, it will be Allow.

Security Groups CLI Example

```
aws ec2 create-security-group \  
--group-name MySecurityGroup \  
--description "My security group" \  
--vpc-id vpc-xxxxxxx
```

Create the security groups

```
aws ec2 authorize-security-group-ingress \  
--group-id sg-xxxxxxx \  
--protocol tcp \  
--port 80 \  
--cidr 0.0.0.0/0
```

Add rules to the security group

```
aws ec2 modify-instance-attribute \  
--instance-id i-xxxxxxxxxxxxxxxxx \  
--groups sg-xxxxxxxxxxxxxxxxx
```

Associate EC2 instance to the security group.

Security Groups – Limits

- You can have **up to 10,000 Security Groups in a Region** (default is 2,500)
- You can have **60 inbound rules** and **60 outbound rules** per security group
- **16 Security Groups** per Elastic Network Interface (ENI) (default is 5)

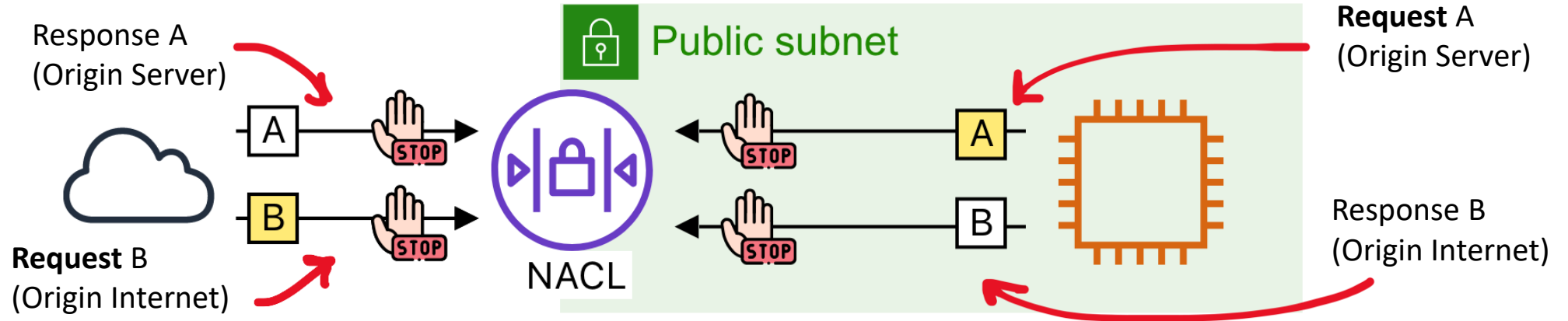
Security groups do not filter traffic destined to and from the following:

- Amazon Domain Name Services (DNS)
- Amazon Dynamic Host Configuration Protocol (DHCP)
- Amazon EC2 instance metadata
- Amazon ECS task metadata endpoints
- License activation for Windows instances
- Amazon Time Sync Service
- Reserved IP addresses used by the default VPC route

Stateful vs Stateless

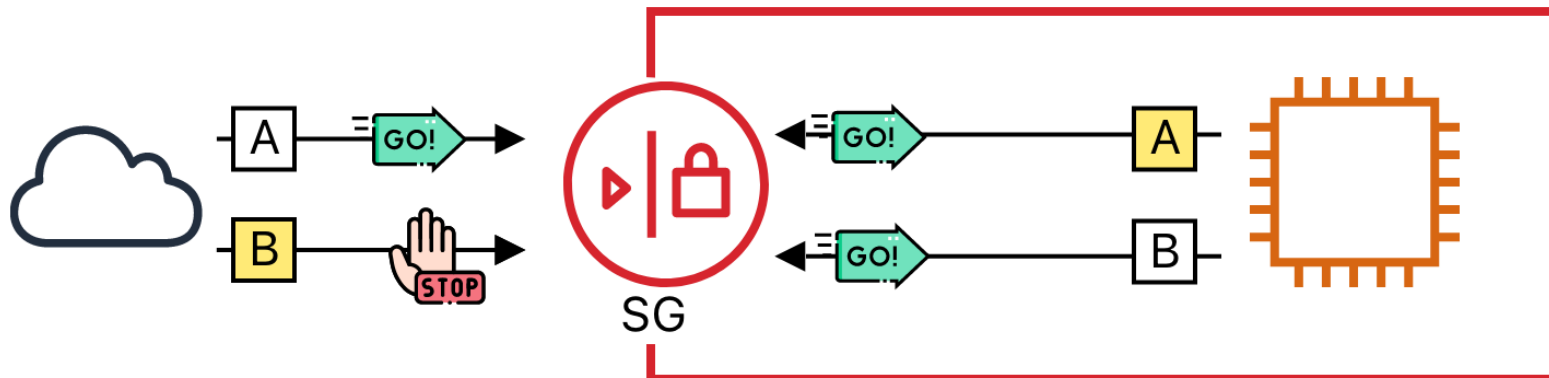
Stateless Firewalls like **AWS NACLs** are **not aware of the state of the request**.

In both directions **they'll treat you like a stranger and stop you both ways** and do a rule check



Stateful Firewalls like **AWS Security Groups** **are aware of the state of the requests**

For SGs they allow all outbound requests, responses for requests that originated outbound are allowed back through





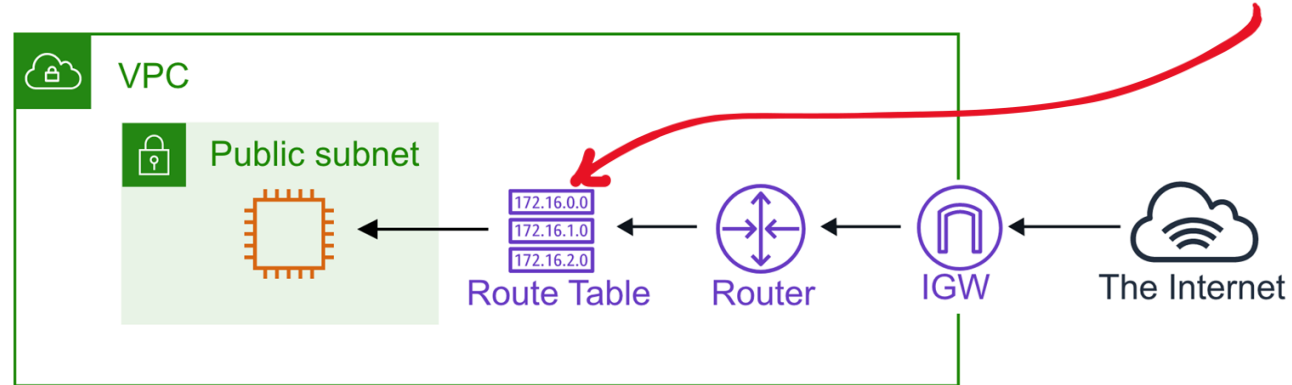
Security Groups CheatSheet

- Security Groups act as a firewall at the instance level
- Unless explicitly allowed, all **inbound traffic** is **blocked by default**.
- Newly created security groups allow all outbound traffic by default, but you can customize or remove the rules to control outbound access.
- You can specify the source to be either an IP range, a single IP Address, or another security group
- Security Groups are **STATEFUL** (if traffic is allowed inbound, it is also allowed outbound)
- Any changes to a Security Group take effect immediately.
- EC2 Instances can belong to multiple security groups
- Security groups can contain multiple EC2 Instances.
- You **cannot block specific IP addresses** with Security Groups. For this, you would need a Network Access Control List (NACL)
- You can have up to 10,000 Security Groups per Region (default 2,500)
- You can have 60 inbound and 60 outbound rules per Security Group
- You can have 16 Security Groups associated with an ENI (default is 5)

Route Tables

172.16.0.0
172.16.1.0
172.16.2.0

Route tables (RT) are used to determine **where network traffic is directed**



Routes (2)			
Filter routes			
Destination ▾	Target ▾	Status ▾	Propagated
0.0.0.0/0	igw-09e4a0e214fe086b9	✓ Active	No
10.0.0.0/16	local	✓ Active	No

A route table contain a set of **routes**

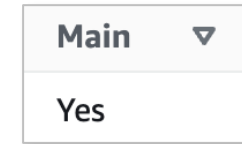
- Each **subnet** in your VPC **must be implicitly or explicitly associated** with a route table
- A subnet can only be associated **with one route table at a time**
 - but you can associate multiple subnets with the same route table.

Route Tables

Main Route Table

A default route table created alongside every VPC which cannot be deleted.

A subnet that is not explicitly associated with a route table will use the Main Route Table



Custom Route Table

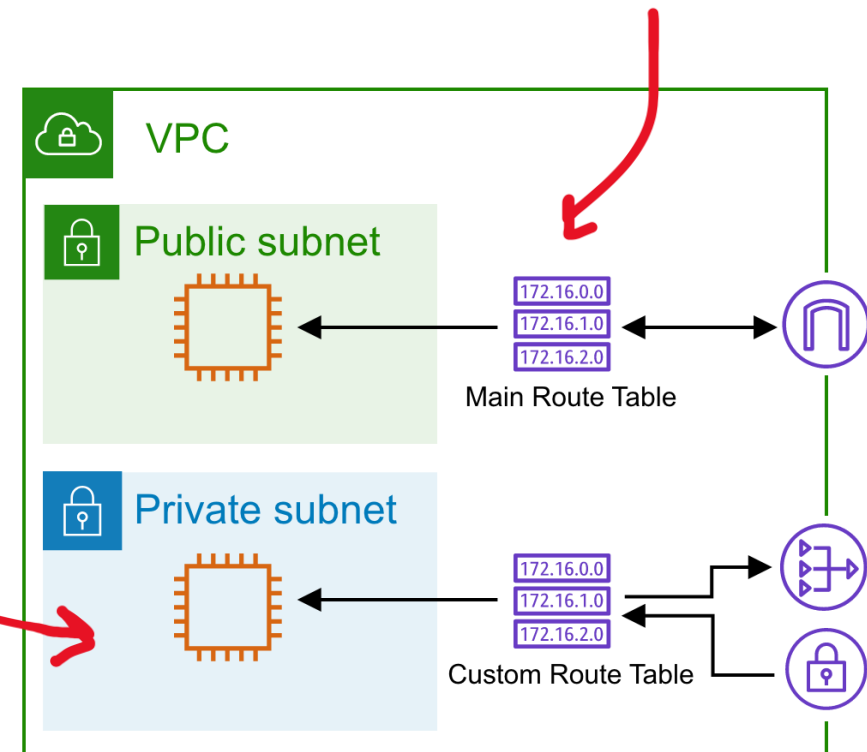
A route table that you can create for your VPC.

Eg. *A custom route table could be used if you needed specific subnets to only route out to a VPN and not the IGW*

Destination

Where the route will go. Here you will specify an IPv4 or IPV6 CIDR Block or Managed Prefix List:

- 0.0.0.0/0
- ::/0
- pl-02cd2c6b (com.amazonaws.us-east-1.dynamodb)



Route Tables

Target

- **Local** — The **default local route** that lets associate subnets within the VPC route to the other route.
- **Internet Gateway (IGW)** ingress and egress connections to the internet for IPv4 and IPv6
- Virtual Private Gateway (VPG) — out to a private connection to a on-premise network
- NAT Gateway — egress connections for private instances out to the internet for IPv4
- Egress Only Internet Gateway — egress connections for private instances out to the internet for IPv6
- Instance — out to a specific EC2 instance
- Network Interface (ENI) — out to a specific Elastic Network Interface
- Carrier Gateway — out to AWS partnered telecom carrier networks via AWS Wavelength
- Core Network — out to managed wide-area networking (WAN) via AWS Cloud WAN
- Gateway Load Balancer Endpoint — out to a Gateway Load Balancer (GWLB); GWLB is for third-party virtual appliances
- Outposts Local Gateway — out to an Outpost, a physical server rack with AWS services in your own datacenter
- Peering Connection — out to another Virtual Private Cloud (VPC)
- Transit Gateway (TGW) — out to a transit hub for connecting multiple VPCs and on-premises network

What is a Gateway?

What is a Gateway?

A gateway (in the context of cloud services) is a networking service which **sits between two different networks**. Gateways often act as reverse proxies, firewall and load balancers.



Internet Gateway

Inbound and Outbound public traffic for IPv4 and IPV6



Egress-Only Internet Gateway

Outbound private traffic for IpV6



Carrier Gateway

Connecting to AWS partnered telecom network



NAT Gateway

Outbound private traffic for IpV4



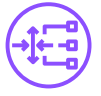
Virtual Private Gateway

The endpoint into your AWS account for a VPN connection



Customer Gateway

The endpoint into your on-premise account for a VPN connection



Gateway Load Balancer (GWLB)

Layer 3 (Network layer) load balancer to run and scale third-party virtual applications eg. Firewalls, IDS/IPS



Direct Connect Gateway

The endpoint connection to a fiber optic connection at co-location data center



AWS Backup Gateway

The endpoint connection for AWS managed backups



IoT Device Gateway

The endpoint connection to send IoT data in both directions



AWS Transit Gateway

Hub and spoke model to simplify VPC peering



Amazon API Gateway

Abstracts API endpoints to services



AWS Storage Gateway

Syncing, caching or extending local storage to cloud storage



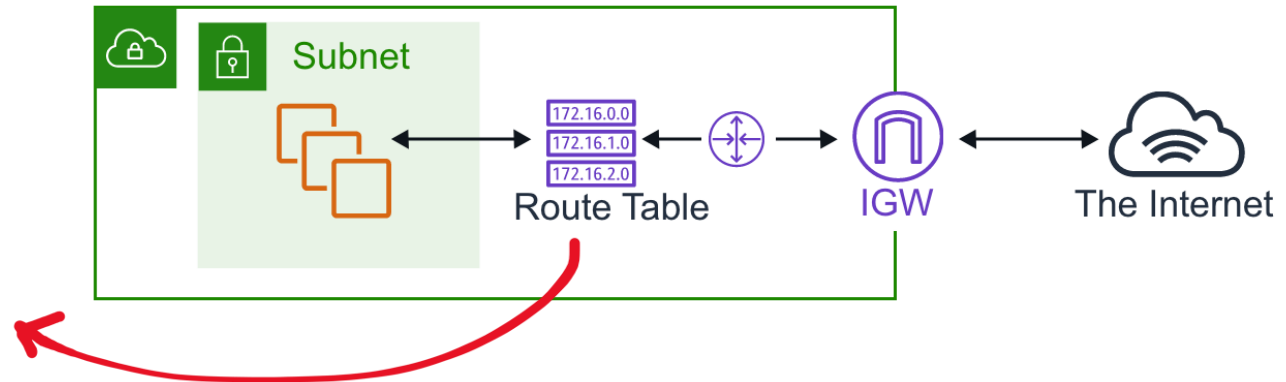
Some cloud services call their gateways "load balancers" or visa versa.

Internet Gateway (IGW)



Internet Gateway (IGW) allows both **inbound and outbound internet traffic** to your VPC

Destination	Target
10.0.0.0/16	local
2001:db8::/64	local
0.0.0.0/0	igw-xxxxxxx
::/0	igw-xxxxxxx



These are **catch-all-routes**.

Always used with IGW to access the internet.

Features of IGW

- It works for both **IPv6 and IPv4 addresses**
- Provides a target in your **VPC route tables** for **internet-routable traffic**
- Public IPv4 addresses do not require NAT, but private IPv4 addresses do
 - NAT is not needed for **IPv6 addresses** because they are all public.

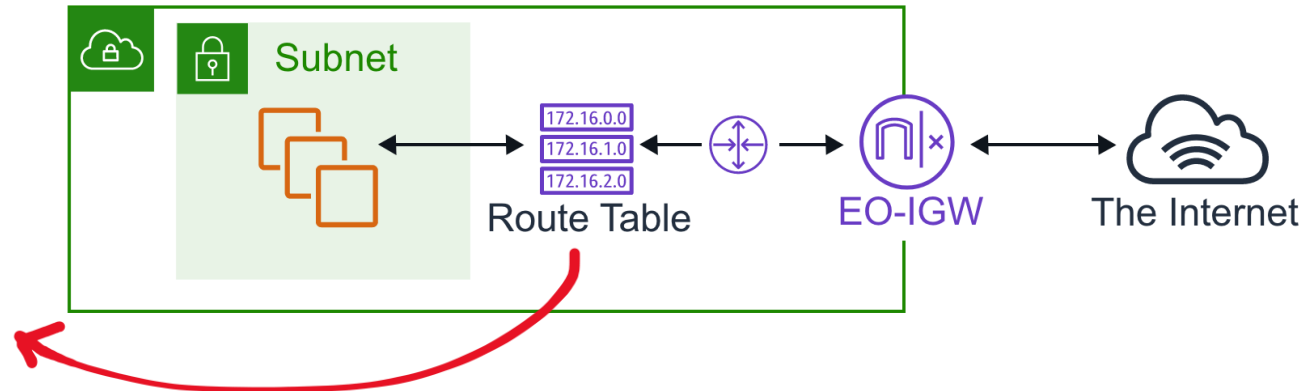
Default VPCs come with an IGW. For **Non-Default VPCs**, you must manually create and associate your IGW

Egress-Only Internet Gateway (EO-IGW)



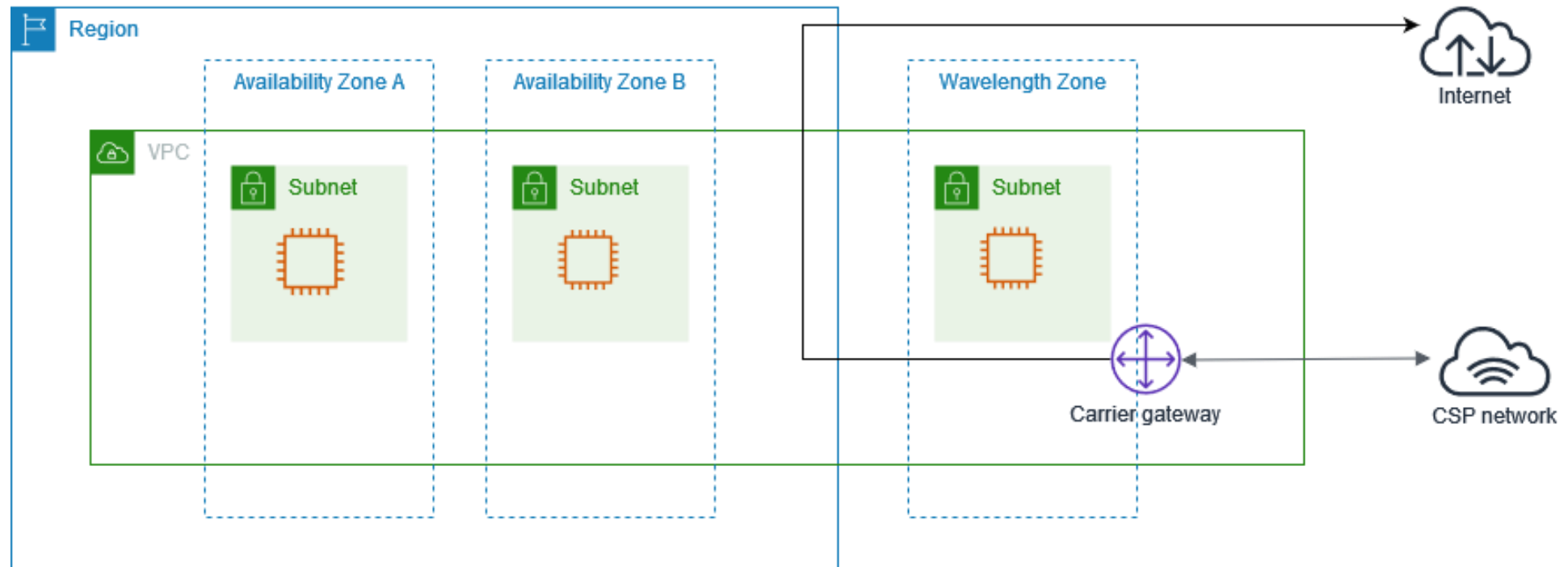
Egress-Only Internet Gateways (EO-IGW) are specifically for IPv6 when you want to allow outbound traffic to the internet **but prevent inbound traffic from the internet.**

Destination	Target
10.0.0.0/16	local
2001:db8::/64	local
::/0	igw-xxxxxxx



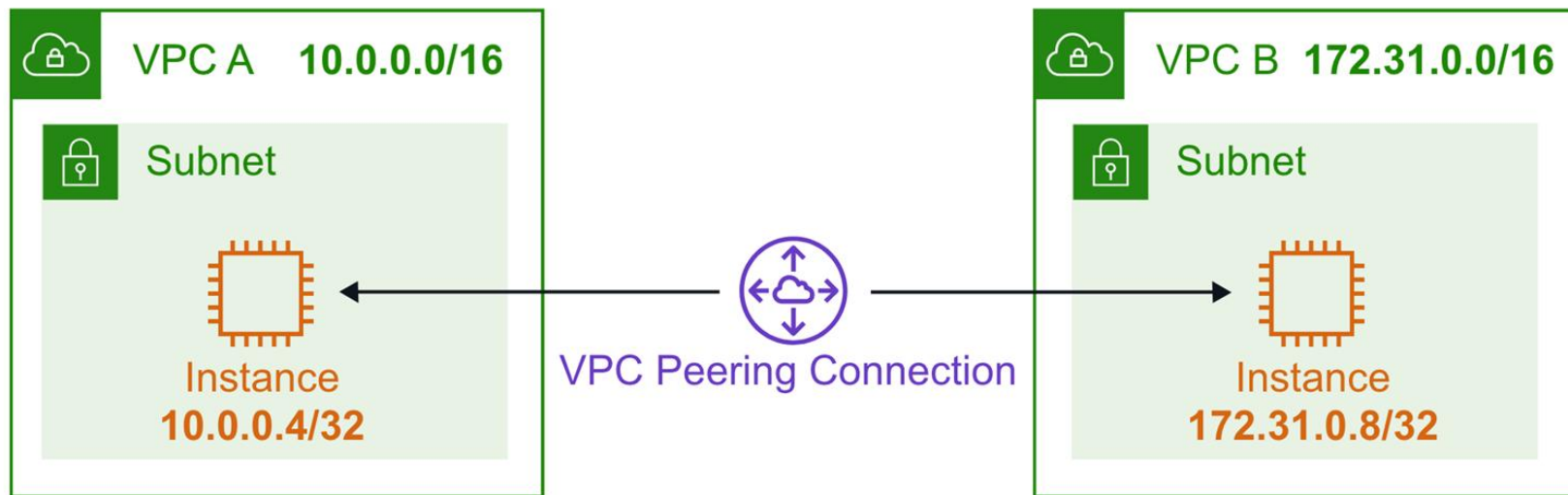
- IPv6 address are all public, and so they don't require Network Address Translation.
- However an IGW would not restrict inbound traffic.
- So an EO-IGW ensure your instances are private by denying inbound traffic.

Carrier Gateway



VPC Peering

VPC Peering allows you to connect one VPC with another over a **direct network route** using **private IP addresses**.



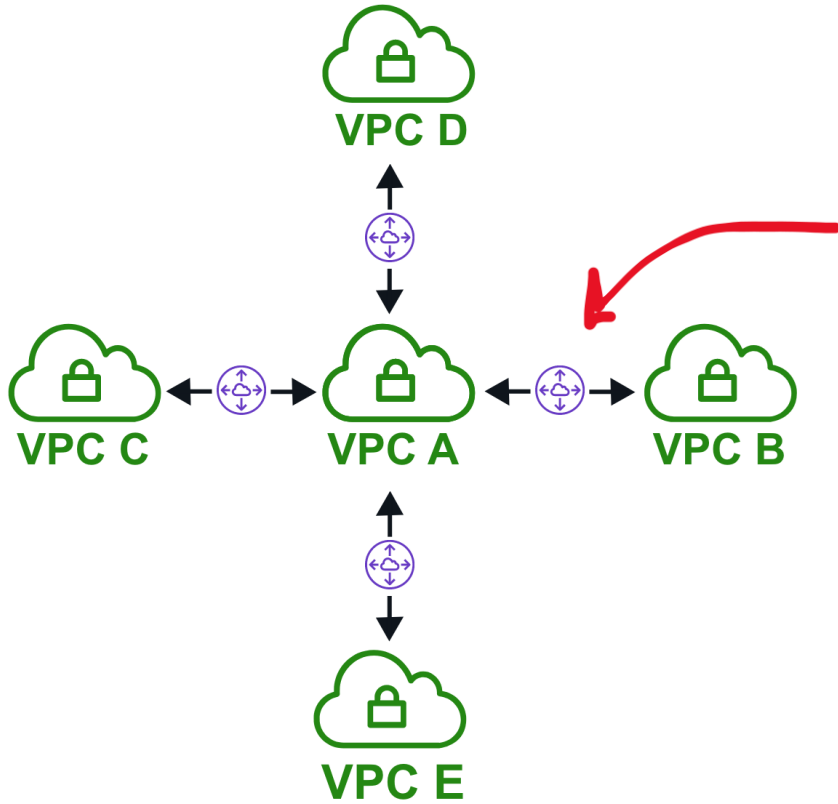
VPC peering connection is not a gateway

VPC peering connection not a VPN connection

VPC peering connection does not rely on a separate piece of physical hardware

There is no single point of failure for communication or a bandwidth bottleneck.

VPC Peering



- VPC peering can be between IPv4 or IPv6 address
- Instances on peered VPCs behave just like they are on the same network
- Connect VPCs across the same or different AWS accounts and regions
- Peering uses a **Star Configuration**: 1 Central VPC - 4 other VPCs
- No Transitive Peering (peering must take place directly between VPCs)
 - Needs a one to one connect to immediate VPC
 - For Transitive peering using **AWS Transit Gateway**
- No Overlapping CIDR Blocks
- Data transfer cross-AZ or cross-Region incurs charges
 - Data transfer within the same AZ is free

VPC Peering

```
aws ec2 create-vpc-peering-connection \  
--vpc-id requester-vpc-id \  
--peer-vpc-id acceptor-vpc-id
```

Create the peering connection

Provide the two VPCs and it will return back a peering connection ID

```
aws ec2 accept-vpc-peering-connection \  
--vpc-peering-connection-id pcx-XXXXXXX
```

Accept the peering connection

Accept the peering connection

```
aws ec2 create-route \  
--route-table-id rtb-requester \  
--destination-cidr-block acceptor-vpc-cidr \  
--vpc-peering-connection-id pcx-XXXXXXX
```

Enable the routing of traffic between the two VPCs

Update your route tables so traffic can flow between the VPCs
You'll need to create a route in each of the VPCs

```
aws ec2 create-route \  
--route-table-id rtb-accepter \  
--destination-cidr-block requester-vpc-cidr \  
--vpc-peering-connection-id pcx-XXXXXXX
```

Referencing SG in Peered VPC

You can update the inbound or outbound rules for your VPC security groups to reference security groups in the peered VPC.



```
aws ec2 describe-security-group-references \
--group-id sg-bbbb2222
```

```
{
  "SecurityGroupsReferenceSet": [
    {
      "ReferencingVpcId": "vpc-aaaaaaaa",
      "GroupId": "sg-bbbb2222",
      "VpcPeeringConnectionId": "pcx-b04deed9"
    }
  ]
}
```

Elastic IPs



Elastic IP (EIP) address in AWS is a static IPv4 address

A Static IP is an address that always stays the same.

Use Case 1

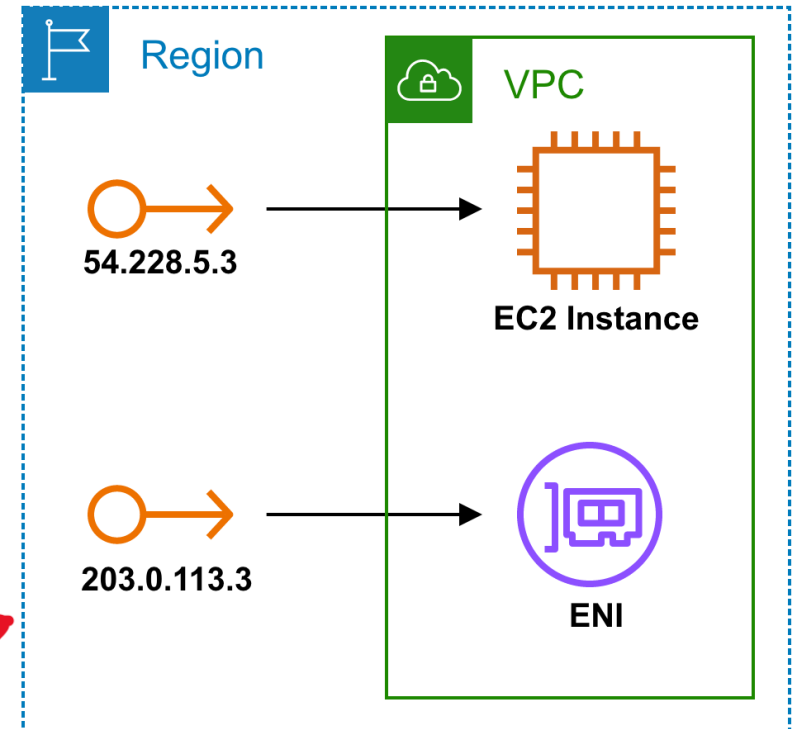
When you restart a EC2 instance, its IP address will change.

This dynamic nature could break external connections relying on the IP address

Use Case 2

When an EC2 instance fails, you might want to use the same IP address on a fail-over instance. An Elastic IP allows you to remap that static IP address

- Elastic IPs are region specific
- Elastic IPs are drawn from Amazon's pool of IPv4 addresses
- Elastic IPs are charge \$1 for each that is allocated and are unassociated.
- Elastic Ips also include the public IPV4 address charge
- Elastic IPs can be associated or unassociated
 - To an **instance** eg. EC2
 - To a **primary network card** eg. ENI



In IPv6, VPC addressing is already globally unique, and therefore Elastic IP addresses are not required.

Elastic IPs

Allocate an EIP

```
aws ec2 allocate-address --domain vpc
{
  "PublicIp": "54.228.5.3",
  "AllocationId": "eipalloc-0a33f63bcede1dff4",
  "PublicIpv4Pool": "amazon",
  "NetworkBorderGroup": "eu-west-1",
  "Domain": "vpc"
}
```

With **--network-board-group**, you can pick very specific Availability Zones, Local Zones or Wavelength Zones.

Associate an EIP

```
aws ec2 associate-address \
  --instance-id i-1234567890abcdef0 \
  --allocation-id eipalloc-0a33f63bcede1dff4
```

We then need to associate the Elastic IP to an instance. You could also use **--network-interface-id** to associate to a network interface

Disassociate an EIP

```
aws ec2 disassociate-address \
  --association-id eipalloc-0a33f63bcede1dff4
```

Deallocate (release) an EIP

```
aws ec2 release-address \
  --allocation-id eipalloc-0a33f63bcede1dff4
```

Elastic IPs

Reassociation

```
aws ec2 associate-address \  
--instance-id i-1234567890abcdef0 \  
--allocation-id eipalloc-0a33f63bcde1dfff4 \  
--allow-reassociation
```



With **--allow-reassociation**, you can tell an address to always attempt to reassociate with the same instance or network interface in the case of failure or restart. You can also tell it to explicitly not to with --no-allow-reassociation

Recover

```
aws ec2 allocate-address \  
--domain vpc \  
--address "54.228.5.3"
```

You can *attempt to (if available)* recover or specific address during allocation.

Custom

```
aws ec2 allocate-address \  
--domain vpc \  
--public-ipv4-pool ipv4pool-ec2-1234567890abcdef0
```

Bring-your-own (BYO) IPV4 address pool, and allocate one from that pool

You have to import a public IPv4 pool in a separate step

Managed Prefix List

Managed Prefix List is a collection of CIDR Blocks to make it easier to configure route tables and security groups

AWS provides by default in each region a set of Managed Prefix Lists you can use:

☐	Prefix list ID ▾	Prefix list name ▾	Max ... ▾	Addr... ▾	State
☐	pl-0025c936f2aacf048	com.amazonaws.ca-central-1.ipv6.route53-healthchecks	-	IPv6	✔ Create-complete
☐	pl-075a07d1abc7dbe5a	com.amazonaws.ca-central-1.vpc-lattice	-	IPv4	✔ Create-complete
☐	pl-078d6f190f393e7fa	com.amazonaws.ca-central-1.route53-healthchecks	-	IPv4	✔ Create-complete
☐	pl-08a6a3685cc410c2d	com.amazonaws.global.groundstation	-	IPv4	✔ Create-complete
☐	pl-0d645b6ffc124ccac	com.amazonaws.ca-central-1.ipv6.vpc-lattice	-	IPv6	✔ Create-complete
☐	pl-38a64351	com.amazonaws.global.cloudfront.origin-facing	-	IPv4	✔ Create-complete
☐	pl-4ea54027	com.amazonaws.ca-central-1.dynamodb	-	IPv4	✔ Create-complete
☐	pl-7da54014	com.amazonaws.ca-central-1.s3	-	IPv4	✔ Create-complete

Entries	Tags
Prefix list entries (5)	
Find entries	
CIDR	
16.12.4.0/23	
3.5.252.0/22	
52.95.145.0/24	
52.95.146.0/23	
52.95.190.0/24	

A prefix list can be either a collection of IPv4 or IPv6.

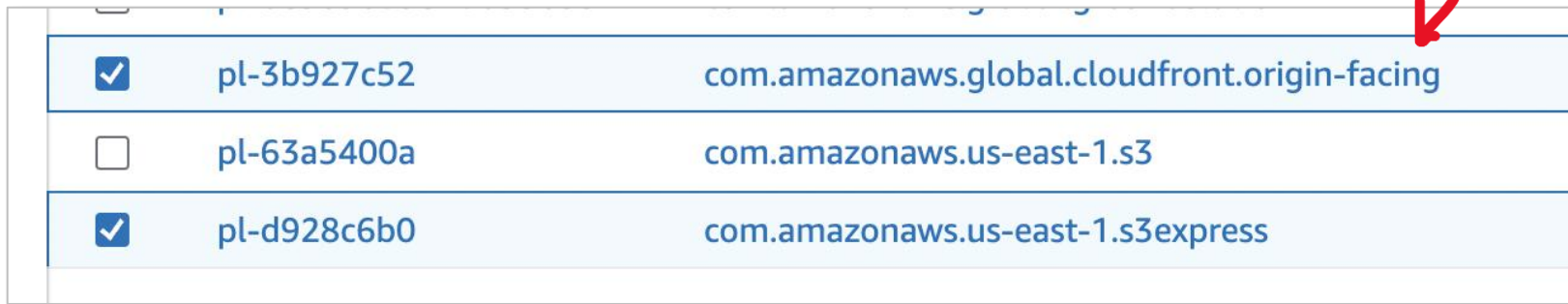
- IPv4 and IPv6 cannot be mixed.

You can create your own prefix list with a collection of CIDR Blocks
An AWS owned prefix list will be updated by AWS.

Managed Prefix List

Some AWS owned managed prefix lists **only appear in specific regions**.

- CloudFront is a global service but its origin default to us-east-1
- S3Express is currently only available in us-east-1 so it doesn't appear anywhere else.



☒	pl-3b927c52	com.amazonaws.global.cloudfront.origin-facing
☐	pl-63a5400a	com.amazonaws.us-east-1.s3
☒	pl-d928c6b0	com.amazonaws.us-east-1.s3express

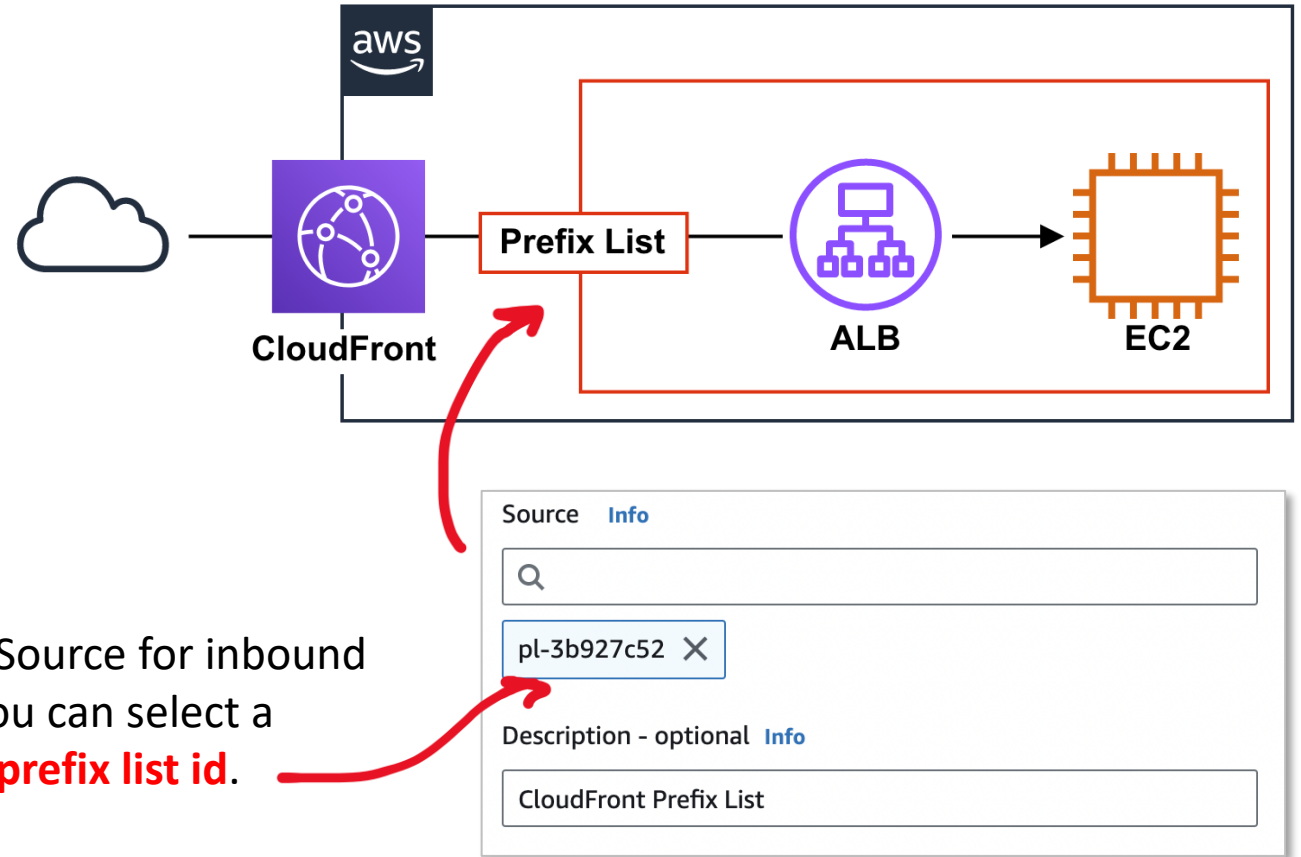
Managed Prefix List

A managed prefix list could be used to only allow CloudFront distributions origins to have inbound access to a security.



*AWS owned Managed Prefix List are including CIDR Block ranges for all AWS accounts. So in this use-case you would be allowing any AWS customer's CloudFront distribution origins.

When you assign a Source for inbound or outbound rule you can select a prefix supplying its **prefix list id**.



AWS IPv6 Support

IPv6 was developed to **provide a solution for the eventual exhaustion of all IPv4 addresses**.
AWS services provides support for IPv6 but its configuration and access will vary per service

2001:0db8:85a3:0000:0000:8a2e:0370:7334

A service will be configured for either:

IPv6 Only

OR

Dual Stack (IPv4 and IPv6)

A service endpoint is the way to access Ipv6:

Public Endpoint Support

AND / OR

Private Endpoint Support

- Note that all AWS services support IPv4.
- Not all services or their resources have IPv6 turned on by default

Migrating from IPV4 to IPV6

IPv4 only VPCs **can be migrated** to operate in Dual Stack mode (Ipv4 and IPv6)

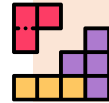
Steps to Migrate a IPv4 Only VPC to dualstack

1. Add new IPv6 CIDR block to VPC
2. Create or Associate IPv6 Subnets
 - IPv4 Subnets can't be migrated
3. Update Route Table for IPv6 to IGW
4. Upgrade SG rules to include IPv6 address ranges
5. Migrate EC2 instance type if it does not support IPV6

You cannot disable IPv4 support for your VPC and subnets; this is the default IP addressing system for Amazon VPC and Amazon EC2.

Network Address Usage

Network Address Usage (NAU) is a metric applied to resources in your virtual network to help you plan for and monitor the size of your VPC



NAU helps you ensure you don't run out of room in your VPC!

NAU **represents specific resources as a NAU unit**



Resource	NAU units
Each IPv4 and IPv6 address assigned to an ENI	1
Additional ENI's attached to EC2 instances	1
Prefix assigned to a network interface	1
Network Load Balancer per AZ	6
VPC endpoint per AZ	6
Transit gateway attachment	6
Lambda function	6
NAT gateway	6
EFS attached to an EC2 instance	6

A VPC or Peered VPCs can handle a certain amount of NAUs

VPC NAU Limit

- 64,000 NAUs (default)
- 256,000 NAUs (quota increase)

Peered VPCs NAU Limit

- 128,000 NAUs (default)
- 512,000 NAUs (quota increase)
- Applies to all VPCs in the peer

Network Address Usage

- You can use NAUs to manually calculate when planning a VPC.
- You can use **CloudWatch NAU metrics** to automatically keep tracks of NAUs.
- You need to **enable NAU monitoring** on a VPC to collect metrics for CloudWatch

Metrics

- AWS/EC2/NetworkAddressUsage
- AWS/EC2/ NetworkAddressUsagePeered
- AWS/Usage/ResourceCount

```
aws ec2 modify-vpc-attribute \  
--vpc-id vpc-xxxxxxx \  
--enable-network-acl-logging Value=true
```

You could set a **CloudWatch Alarm** to alert to monitor changes in your NAU

```
aws cloudwatch put-metric-alarm \  
--cli-input-json file://nau-alarm.json
```

```
{  
  "Namespace": "AWS/EC2",  
  "MetricName": "NetworkAddressUsage",  
  "Dimensions": [{  
    "Name": "Per-VPC Metrics",  
    "Value": "vpc-0123456798"  
  }],  
  "AlarmActions": ["arn:aws:sns:us-west-1:123456789012:mytopic"],  
  "ComparisonOperator": "GreaterThanThreshold",  
  "Period": 86400,  
  "EvaluationPeriods": 1,  
  "Threshold": 50000,  
  "AlarmDescription": "50k NAUs as the threshold",  
  "AlarmName": "VPC NAU Utilization",  
  "Statistic": "Maximum"  
}
```

Shared VPCs



AWS Resource Access Manager (RAM) allows you to share resources across your AWS Accounts.

VPCs can be shared with other AWS Accounts within the same account to centrally manage resources in a single VPC.

Using a shared VPC allows you to:

- reduces the number of VPCs that you create and manage
- separate accounts for billing and access control
- You share VPCs by sharing subnets
- You can share only non-default subnets
- You need to create a resource share in RAM (what you're sharing)
- You need to create a shared principles in RAM (who you're sharing with)

In order to share VPCs you need to enable sharing within your AWS Organization via the **RAM API**

```
aws ram enable-sharing-with-aws-organization
```

Owner ID

 655604346524
(shared)

Shared VPCs will appear in the shared account with the specific share subnets. You can tell if its shared by checking the OwnerID.

Shared VPCs

Sharing a subnet with other accounts will allow them to deploy resources.

You can share non-default subnets by **creating a resource share**.

```
aws ram create-resource-share \  
--name "ShareSubnet" \  
--resource-arns "arn:aws:ec2:region:account-id:subnet/subnet-id" \  
--principals "0123456789012"
```



The principle is the account ID of the account you are sharing with.

<this seems complicated>

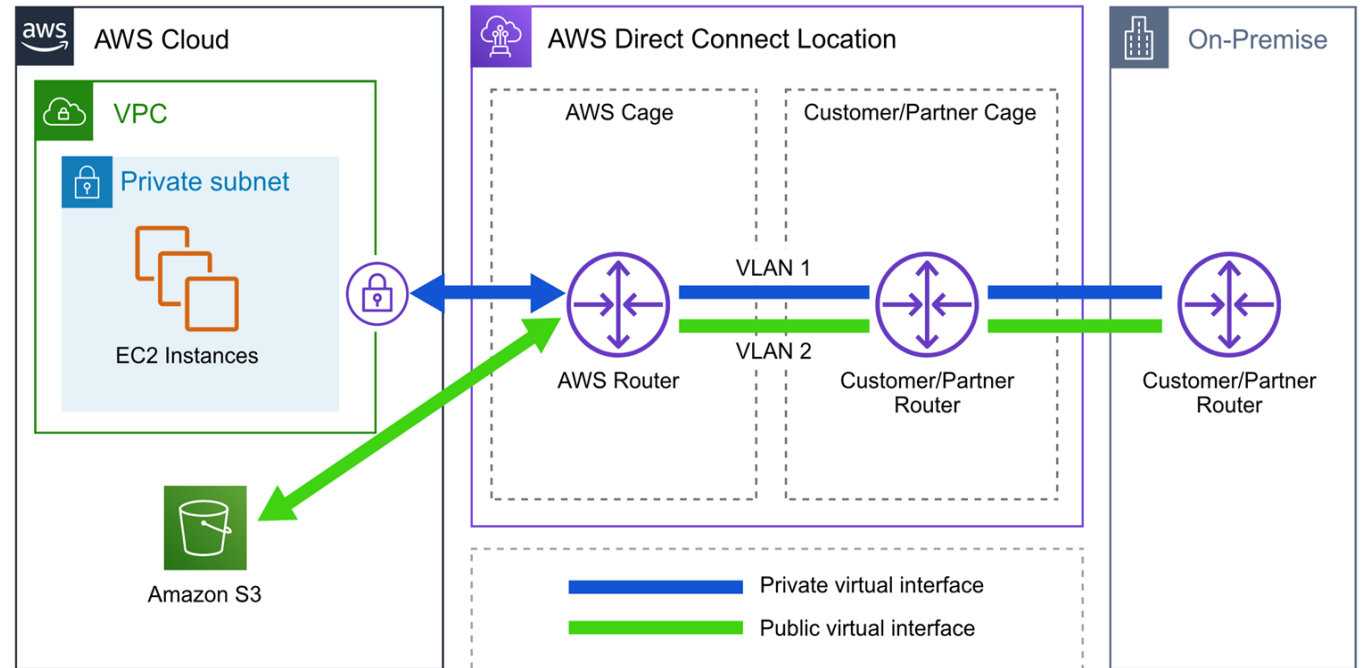
AWS Direct Connect



AWS Direct Connect is the AWS solution for establishing **dedicated network** connections from on-premises locations to AWS.

Direct Connect has two **very-fast network** connection options:

1. Lower Bandwidth **50MBps-500MBps**
2. Higher Bandwidth **1GBps, 10GBps 100GBps**



Helps **reduce network costs** and **increase bandwidth throughput**. (great for high-traffic networks)

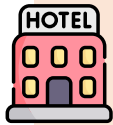


Provides a **more consistent network experience** than a typical internet-based connection. (reliable and secure)

AWS Direct Connect

Connection Requirements:

1. Your network is **co-located** with an existing AWS Direct Connect location
2. You are working with an AWS Direct Connect partner who is a member of the AWS Partner Network (APN).
3. You are working with an independent service provider to connect to AWS Direct Connect



A co-location (aka carrier-hotel) is a data center where equipment, space, and bandwidth are available for rental to retail customers

Network Requirements:

- Your network must use single-mode fiber with either a:
 - 1000BASE-LX (1310 nm) transceiver for 1 gigabit Ethernet
 - 10GBASE-LR (1310 nm) transceiver for 10 gigabit
 - **100GBASE-LR4** for 100 gigabit Ethernet.
- Auto-negotiation for a port must be disabled for a connection with a port speed of more than 1 Gbps
 - For 1Gbps speed auto-negotiation may or may not need to be disabled
- 802.1Q VLAN encapsulation must be supported across the entire connection, including intermediate devices.
- Your device must support Border Gateway Protocol (BGP) and BGP MD5 authentication
- (Optional) You can configure Bidirectional Forwarding Detection (BFD) on your network.



AWS Direct Connect

Direct Connect Supports:

- both IPv4 and IPv6
- Ethernet frame size of 1522 or 9023 bytes
 - (14 bytes Ethernet header + 4 bytes VLAN tag + bytes for the IP datagram + 4 bytes FCS) at the link layer

Direct Connect Maintenance:

Periodically AWS performance maintenance on the hardware fleet.

There are two types of maintenance:



1. Planned

- scheduled in advance to improve availability and deliver new features
- Three notifications are provided 1, 5 and 10 calendars in advanced



2. Emergency

- Maintenance that isn't planned in advance (due to unexpected failure)
- Impacted customers are notified 60 mins prior to maintenance

AWS Direct Connect

There are **three factors to pricing**:

1. Capacity (port size)

- the maximum rate that data can be transferred through a network connection
- The larger the port size the greater their cost eg. 50,100,200,300,400,500Mbps ... 1,2,5, 10Gbps

2. Port hours

- for amount of time a Direct Connect port is provisioned for use, **regardless of data transfer**.
- There are two connection types:
 1. Dedicated – physical connections between your network port and an AWS network port
 - Billed per hour via AWS
 2. Hosted – logical connections between AWS partner and AWS network port.
 - Billing is subject to the AWS Direct Connect Deliver Partner

3. Data Transfer Out (DTO)

- Charged based on outbound traffic sent through Direct Connect to destinations outside of AWS
- Data transfer in is free
- Data transfer within the same region will not result in a cost

VPC Endpoints



VPC Endpoints allow you to **privately connect your VPC to other AWS and other services**



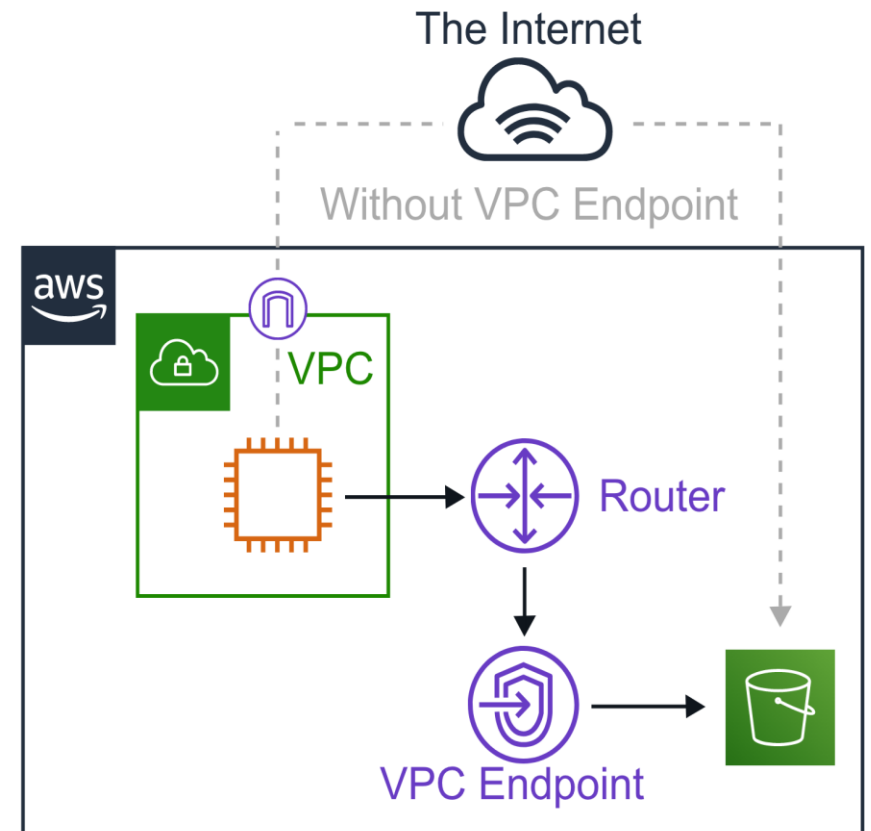
Think of a secret tunnel where you don't have to leave the AWS network

VPC Endpoints provide the following benefits

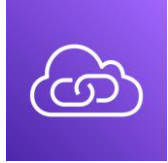
- Eliminates the need for:
 - Internet Gateway (IGW)
 - NAT device
 - VPN connection
 - AWS Direct Connect
- **Instances in the VPC do not require a public IPv4 address**
- **Traffic between your VPC and other services does not leave the AWS network.**
- **Horizontally scaled, redundant, and highly available** VPC component.
- Allows secure communication between instances and services **without adding availability risks or bandwidth constraints** on your traffic.

There are **3 Types** of VPC Endpoints

1. Interface Endpoints
2. Gateway Endpoints
3. Gateway Load Balancer Endpoints



Introduction to PrivateLink

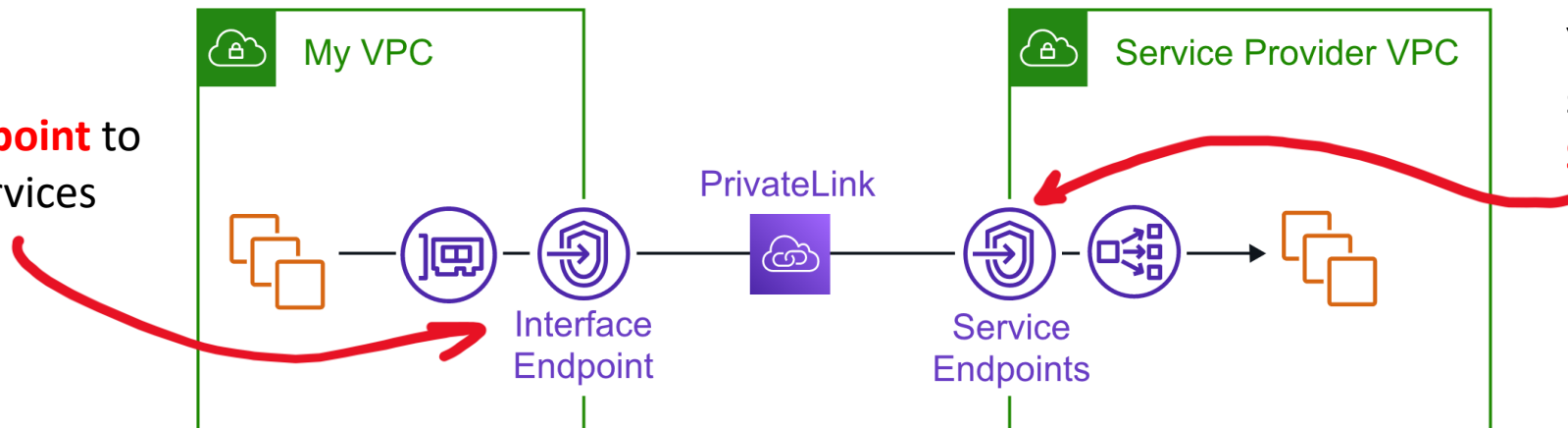


AWS PrivateLink is a broader service that allows you to **securely connect your VPC** to:

- Supported AWS Services
- AWS services hosted in other AWS Accounts
- Supported AWS Marketplace partner services

Enables secure connections **without** the need for an IGW, NAT, VPN, or AWS Direct Connect connection

You create an **Interface Endpoint** to connect to services



You can create your own services by creating a **Service Endpoint**

PrivateLink Ready partner services allows you to access SaaS products privately, as if they were running in your own VPC

Interface Endpoints



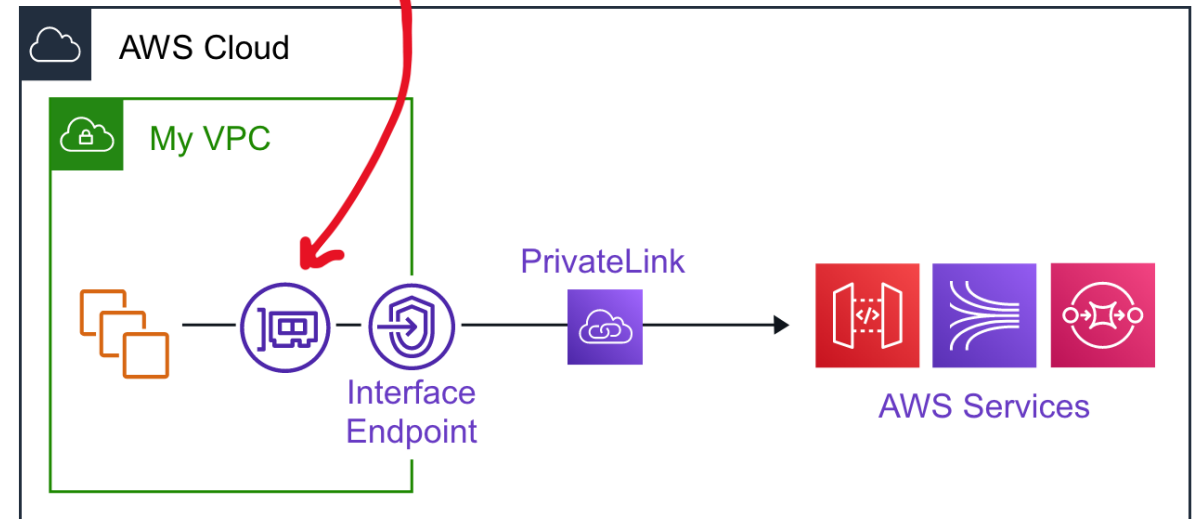
Interface Endpoints are **Elastic Network Interfaces (ENI)** with a **private IP address**. They serve as an entry point for traffic going to a supported service.

Access services hosted on AWS easily and securely by keeping your network traffic within the AWS network

Interface Endpoints support the following AWS Services...

- API Gateway
- CloudFormation
- CloudWatch
- Kinesis
- SageMaker
- Codebuild
- AWS Config
- EC2 API
- ELB API
- AWS KMS
- Secrets Manager
- Security Token Service
- Service Catalog
- SNS
- SQS
- Systems Manager
- Marketplace Partner Services
- Endpoint Services in other AWS accounts

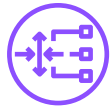
Elastic Network Interface (ENI) is an entry point for traffic destined to the service.



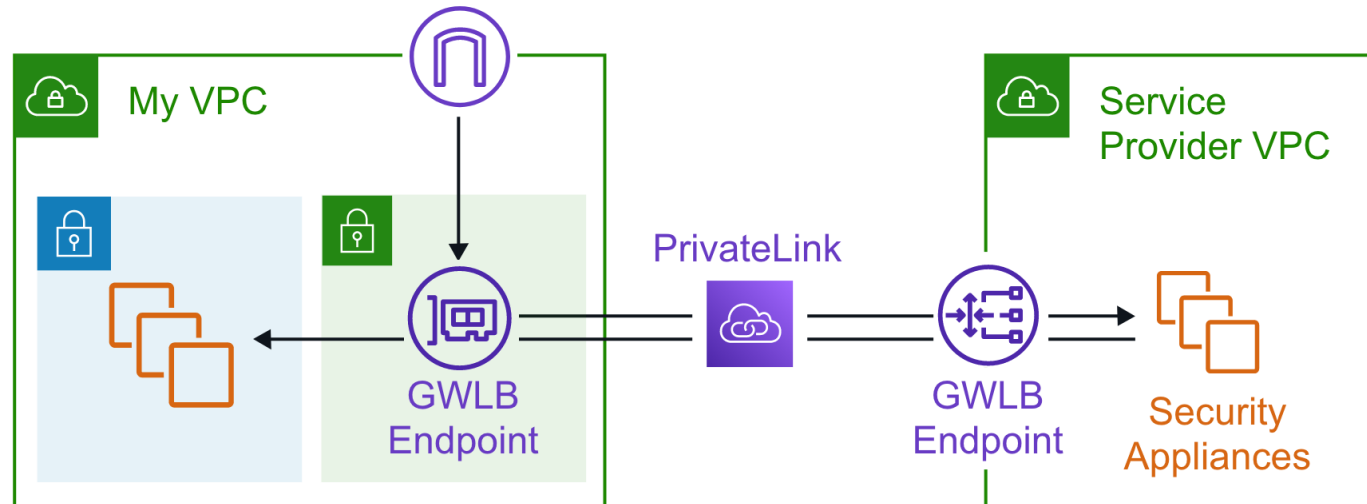
Pricing per VPC endpoint per AZ (\$/hour) **0.01**
Pricing per GB data processed (\$) **0.01**

~\$7.5 / mo

Gateway Load Balancer Endpoints



Gateway Load Balancer (GWLB) Endpoints powered via PrivateLink allows you to distribute traffic to a fleet of network virtual appliances.



Deploy, scale and manage:

- Firewalls
- Intrusion Detection and Prevention Systems (IDS/IPS)
- Deep Packet Inspection Systems

You can obtain virtual appliances as-a-service from:

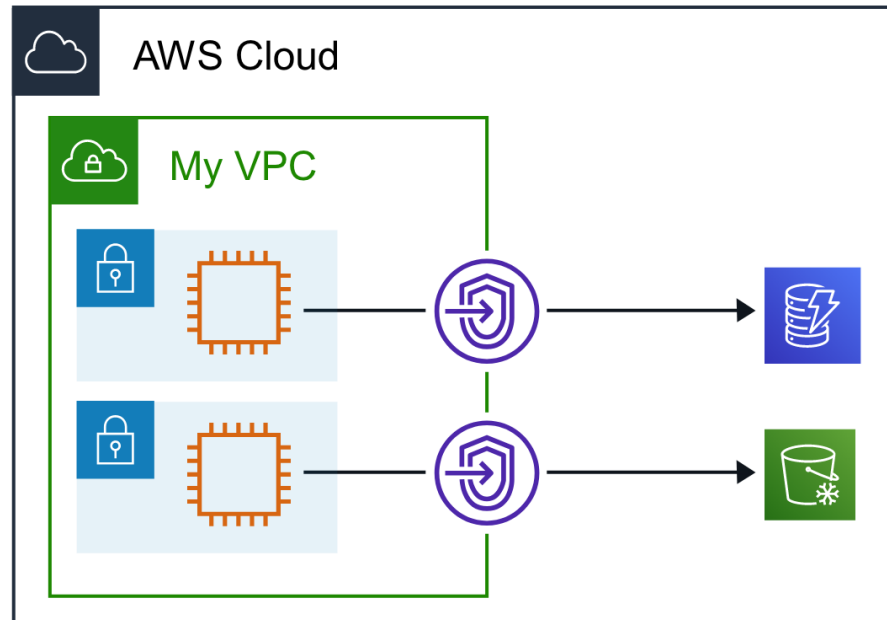
- AWS Partner Network (APN)
- AWS Marketplace

You can send traffic to GWLB by making simple configuration updates in your **VPCs' route tables**

VPC Gateway Endpoints



A **Gateway Endpoint** provide reliable connectivity to Amazon S3 and DynamoDB without requiring an internet gateway or a NAT device for your VPC.



Gateway endpoints do not use AWS PrivateLink
Gateway endpoints have no additional charge
Gateway endpoints support the following services:

- Amazon DynamoDB
- Amazon S3

To create a Gateway Endpoint, you must specify the VPC in which you want to create the endpoint and the service to which you want to establish the connection.

VPC Endpoints Comparison

Feature	Interface Endpoint	Gateway Endpoint	GWLB Endpoint
Type	Elastic Network Interface (ENI)	Gateway with VPC	Type of Interface Endpoint
Use Case	Private connection to AWS services, partner services, and other VPCs without public IPs.	Private connections to S3 and DynamoDB from VPC	Route traffic to third-party virtual appliances like firewalls in another VPC.
Service Integration	AWS PrivateLink	S3 and DynamoDB	AWS PrivateLink and GWLB
Supported Services	Many AWS Services	S3 and DynamoDB	Third-party virtual applications
Pricing	• Per hour when provisioned • Data processed	• Free	• Endpoint hours • Data processed
Routing Mechanism	DNS interception and routing	Route table entries for specific destinations	Integrates with GWLB
Traffic Direction	Bidirectional	Unidirectional	Usually Unidirectional



VPC Endpoint CheatSheet

- VPC Endpoints help keep traffic between AWS services **within the AWS Network**
- There are two kinds of VPC Endpoints. Interface Endpoints and Gateway Endpoints
- Interface Endpoints **cost money**, Gateway Endpoints **are free**
- Interface Endpoints use an Elastic Network Interface (ENI) with Private IP (powered by AWS PrivateLink)
- Gateway Endpoints is a target for a specific route in your route table
- Interface Endpoints support many AWS services
- Gateway Endpoint only supports DynamoDB and S3

VPC Flow Logs



VPC Flow Logs allow you to **capture IP traffic information** through your VPC.

```
aws ec2 create-flow-logs \  
  --resource-type VPC \  
  --resource-ids vpc-xxxxxxx \  
  --traffic-type ALL \  
  --log-destination-type cloud-watch-logs \  
  --log-destination arn:aws:logs:region:account-id:log-group:log-group-name \  
  --deliver-logs-permission-arn arn:aws:iam::account-id:role/role-name
```

Flow Logs be scoped for the following:

- VPC, Subnets
- Elastic Network Interface (ENIs)
- Transit Gateway
- Transit Gateway Attachment

You can monitor traffic for:

- ACCEPT – traffic was accepted
- REJECT – traffic that was rejected
- ALL — all accepted and rejected traffic

Logs can be delivered to either:

- Amazon S3 bucket
- CloudWatch Logs
- Kinesis Data Firehose

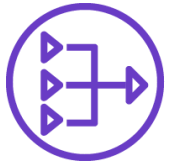
Format of log entry for VPC Flow Log

```
<version> <account-id> <interface-id> <srcaddr> <dstaddr> <srcport> <dstport> <protocol>  
<packets> <bytes> <start> <end> <action> <log-status>
```

```
2 123456789010 eni-abc123de 172.31.16.139 172.31.16.21 20641 22 6 20 4249 1418530010 1418530070  
ACCEPT OK
```

VPC Flow Logs

version	The VPC Flow Logs version.
account-id	The AWS account ID for the flow log.
interface-id	The ID of the network interface for which the traffic is recorded.
srcaddr	The source IPv4 or IPv6 address . The IPv4 address of the network interface is always its private IPv4 address.
dstaddr	The destination IPv4 or IPv6 address . The IPv4 address of the network interface is always its private IPv4 address.
srcport	The source port of the traffic.
dstport	The destination port of the traffic.
protocol	The IANA protocol number of the traffic. For more information, see Assigned Internet Protocol Numbers.
packets	The number of packets transferred during the capture window.
bytes	The number of bytes transferred during the capture window.
start	The time, in Unix seconds, of the start of the capture window.
end	The time, in Unix seconds, of the end of the capture window.
action	The action associated with the traffic: - ACCEPT: The recorded traffic was permitted by the security groups or network ACLs. - REJECT: The recorded traffic was not permitted by the security groups or network ACLs.
log-status	The logging status of the flow log: - OK: Data is logging normally to the chosen destinations. - NODATA: There was no network traffic to or from the network interface during the capture window. - SKIPDATA: Some flow log records were skipped during the capture window. This may be because of an internal capacity constraint, or an internal error.



VPC Flow Logs CheatSheet

- **VPC Flow Logs** monitor the in and out traffic of your Network Interfaces within your VPC
- You can turn on Flow Logs at the VPC, Subnet, or Network Interface level
- VPC Flow Logs **cannot be tagged** like other AWS resources
- You **cannot change the configuration** of a flow log **after it's created**.
- You **cannot enable** flow logs for VPCs which are peered with your VPC **unless it is in the same account**
- VPC Flow Logs can be delivered to an **S3** or **CloudWatch Logs**
- VPC Flow Logs contain the source and destination **IP addresses** (not hostnames)
- In some instances, traffic is **not monitored**:
 - Instance traffic generated by contacting the AWS DNS servers
 - Windows license activation traffic from instances
 - Traffic to and from the instance metadata address (169.254.169.254)
 - DHCP Traffic
 - Any traffic to the reserved IP address of the default VPC router

AWS Virtual Private Network



AWS VPN lets you establish a **secure** and **private tunnel** from your network or device to the AWS global network

AWS Site-to-Site VPN

securely connect on-premises
network or branch office site to VPC

AWS Client VPN

securely connect users to AWS or
on-premises networks

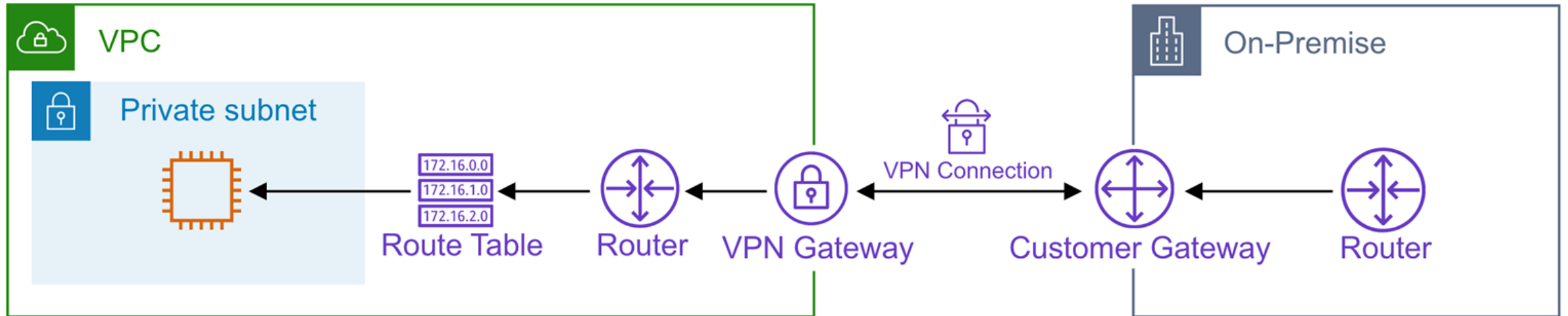
What is IPsec?

Internet Protocol Security (IPsec) is a secure network protocol suite that authenticates and encrypts the packets of data to provide secure encrypted communication between two computers over an Internet Protocol network. It is used in virtual private networks (VPNs)



AWS Site-to-Site VPN

AWS Site-to-Site VPN allows you connect your VPC to your on-premise network.



AWS Site-to-Site has the following components:

- VPN Connection — **secure** connection between VPC and on-premises equipment
- VPN tunnel — **encrypted** connection for your data
- Customer gateway (**CGW**)—provides information to AWS about your customer gateway device
- Customer gateway device — A physical device or software application on your side of the Site-to-Site VPN connection.
- Target gateway — A generic term for the VPN endpoint on the Amazon side of the Site-to-Site VPN connection.
- Virtual private gateway (**VGW**) —VPN endpoint on the Amazon side of your Site-to-Site VPN connection that can be attached to a single VPC
- Transit gateway — A transit hub that can be used to interconnect multiple VPCs and on-premises networks, and as a VPN endpoint for the Amazon side of the Site-to-Site VPN connection.

AWS Site-to-Site VPN

Features:

- Internet Key Exchange version 2 (IKEv2)
- NAT traversal
- 4-byte ASN in the range of 1 – 2147483647 for VGW configuration
- 2-byte ASN for CGW in the range of 1 – 65535.
- CloudWatch metrics
- Reusable IP addresses for your customer gateways
- Additional encryption options
 - AES 256-bit encryption
 - SHA-2 hashing,
 - additional Diffie-Hellman groups
 - Configurable tunnel options
- Custom private ASN for the Amazon side of a BGP session
- Support for IPv6 traffic for VPN connections on a transit gateway
- You can optionally enable acceleration for your Site-to-Site VPN connection via **AWS Global Accelerator**
- You can attach your Site-to-Site VPN to **AWS Cloud WAN**
- You can attach your Site-to-Site VPN to **AWS Transit Gateway**

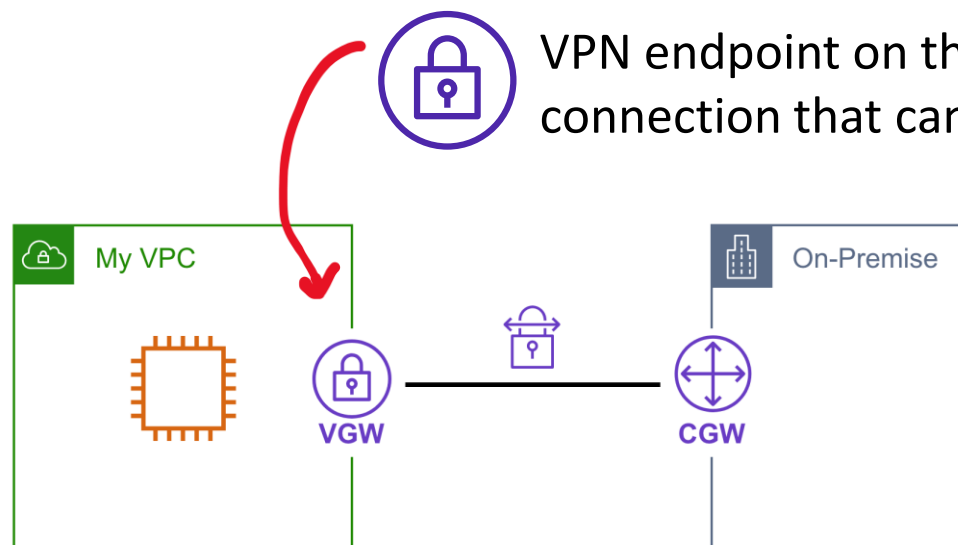
Pricing:

- Each VPN connection hour
- data transfer out from Amazon EC2 to the internet.

Limitations:

- **IPv6 traffic is not supported** for VPN connections on a virtual private gateway.
- An AWS VPN connection does not support Path MTU Discovery.
- recommend that you use non-overlapping CIDR blocks for your networks

Virtual Private Gateway (VGW)



VPN endpoint on the Amazon side of your Site-to-Site VPN connection that can be attached to a single VPC

When you create a VGW you need to assign an Amazon **Autonomous System Number (ASN)** or custom ASN

If you don't specify an ASN, the virtual private gateway is created with the default ASN (64512)

Amazon ASN

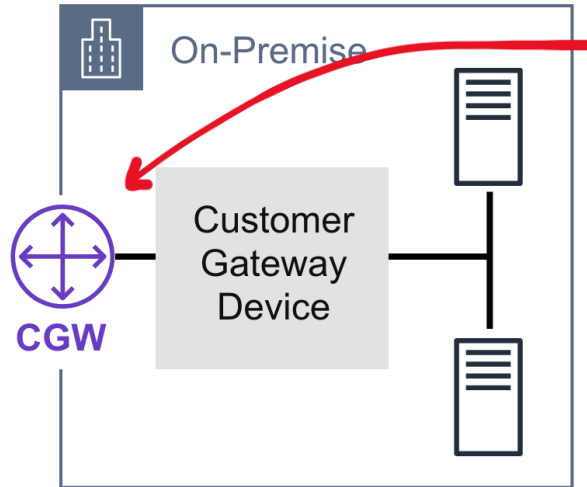
64512

Once you create a VGW you can't change the ASN

What is an ASN?

An Autonomous System Number (ASN) is a unique identifier that is globally allocated to each autonomous system (AS) that participates in the Internet.

Customer Gateway (CGW)



A **Customer Gateway** is a resource that you create in AWS that represents the customer gateway device in your on-premises network.

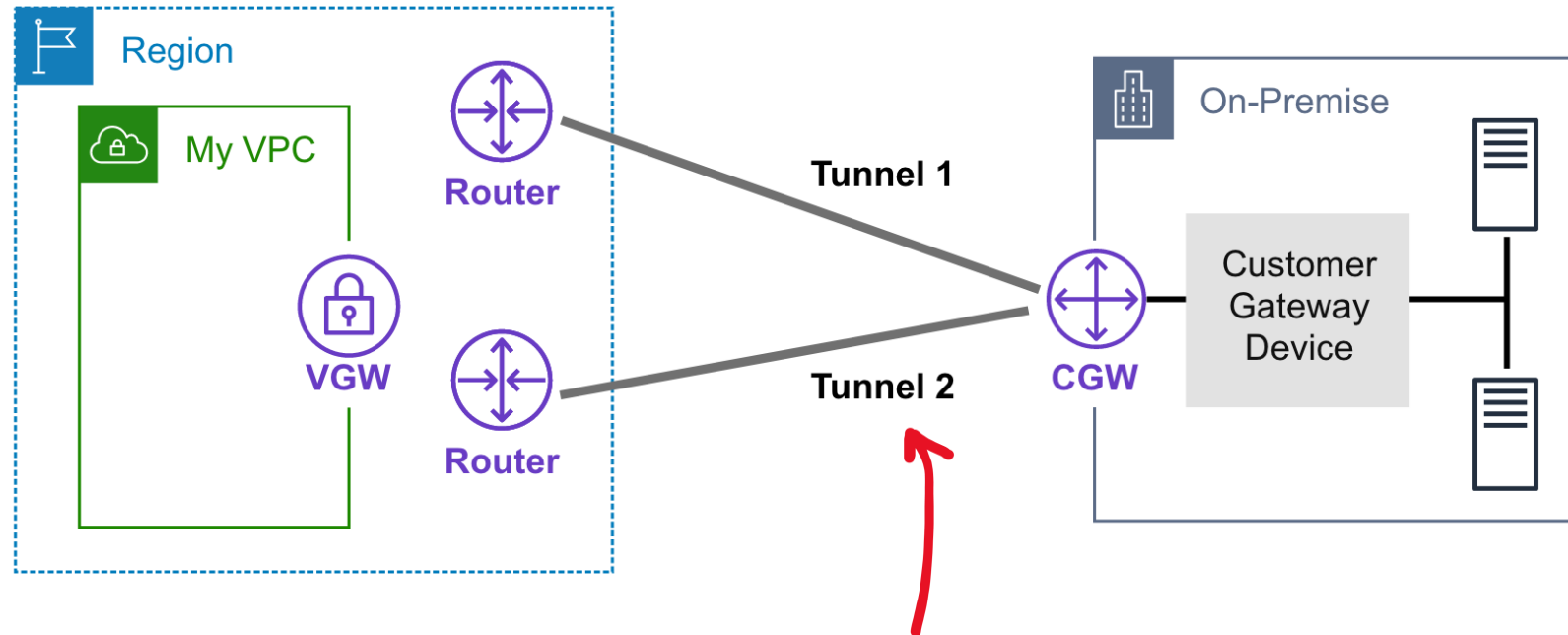
When you configure your CGW you'll set:

- BPG ASN for your customer gateway device
- IP address of customer gateway external device
- Private certificate provisioned by AWS Certificate Manager (ACM)



You will also need to provide additional configuration to your customer gateway device which will establish the connect between AWS and your on-premise network.

Customer Gateway (CGW)



If there's a device failure within AWS, your VPN connection automatically fails over to the second tunnel so that your access isn't interrupted. From time to time, AWS also performs routine maintenance on the VPN connection,

Customer Gateway (CGW)

AWS provides sample configuration files for various customer gateway devices:



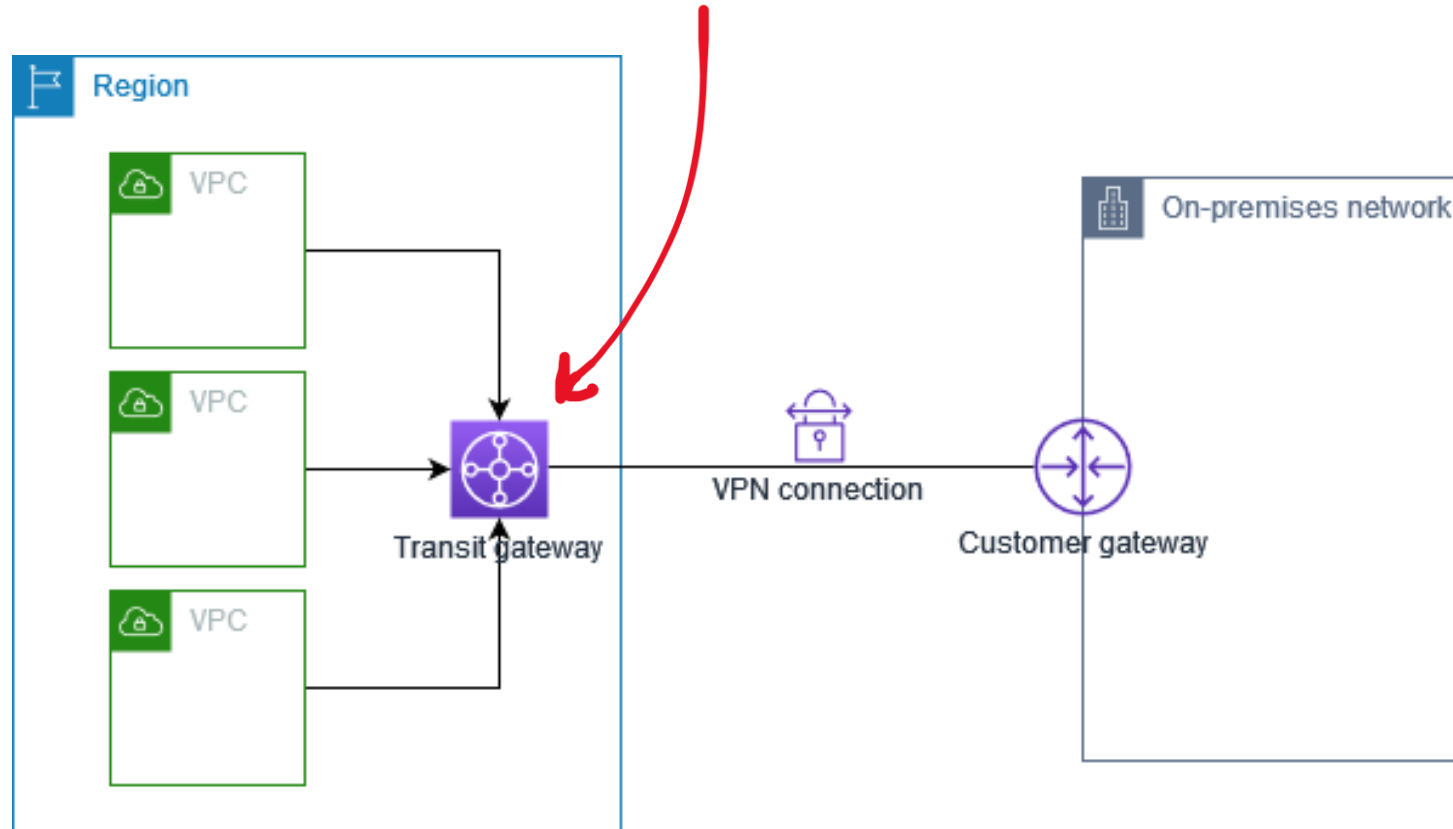
If you have a device **not in the list** you'll have to ensure your device can be configured for the following:

- Internet key exchange (IKE) association
- IPsec security association
- Tunnel Interface
- (Optional) Border Gateway Protocol (BGP) peering

Vendor	Platform	Software
Checkpoint	Gaia	R80.10+
Cisco Meraki	MX Series	15.12+ (WebUI)
Cisco Systems, Inc.	ASA 5500 Series	ASA 9.7+ VTI
Cisco Systems, Inc.	CSRv AMI	IOS 12.4+
Fortinet	Fortigate 40+ Series	FortiOS 6.4.4+ (GUI)
Juniper Networks, Inc.	J-Series Routers	JunOS 9.5+
Juniper Networks, Inc.	SRX Routers	JunOS 11.0+
Mikrotik	RouterOS	6.44.3
Palo Alto Networks	PA Series	PANOS 7.0+
SonicWall	NSA, TZ	OS 6.5
Sophos	Sophos Firewall	v19+
Strongswan	Ubuntu 16.04	Strongswan 5.5.1+
Yamaha	RTX Routers	Rev.10.01.16+

Transit Gateway (TGW)

A transit gateway is a **transit hub** that you can use to interconnect your VPCs and your on-premises networks



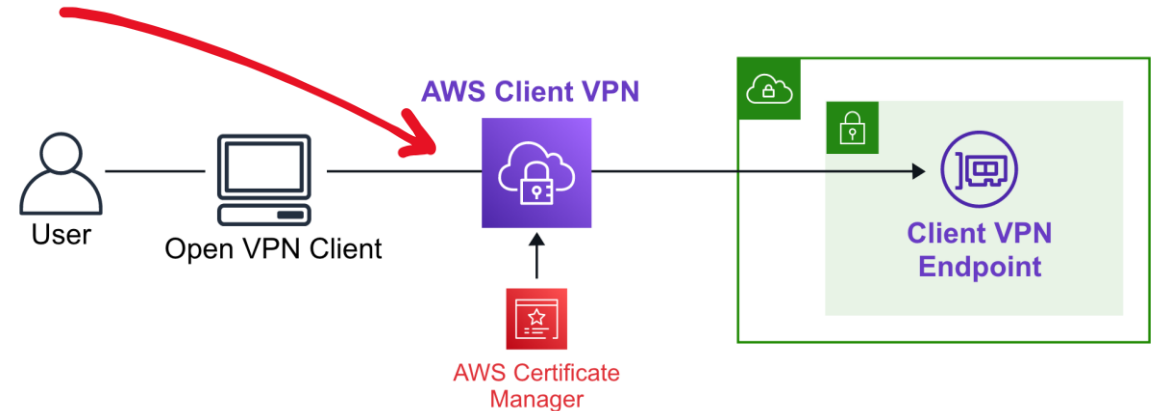
Transit gateway can **support either IPv4 traffic or IPv6 traffic** inside the VPN tunnels

AWS Client VPN



AWS Client VPN is a fully managed client-based VPN service that enables you to **securely access AWS resources** and resources in your on-premises network.

- Certificate based authentication (Mutual authentication)
- Active Directory authentication (AWS Directory Service)
- Federation authentication (Single-Sign-On SAML)
- Uses a single tunnel
- Use Security Groups for granular control
- Use AD Groups for granular control
- Self-service portal to download AWS VPN Desktop Client



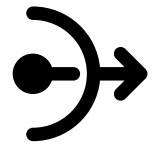
AWS VPN Client has two roles:

- Administrators — responsible for setting up and configuring the service
- Clients — the person who connects to the Client VPN endpoint to establish a VPN session



You could use AWS Client VPN to securely connect to an RDS Instance that is only in a private subnet

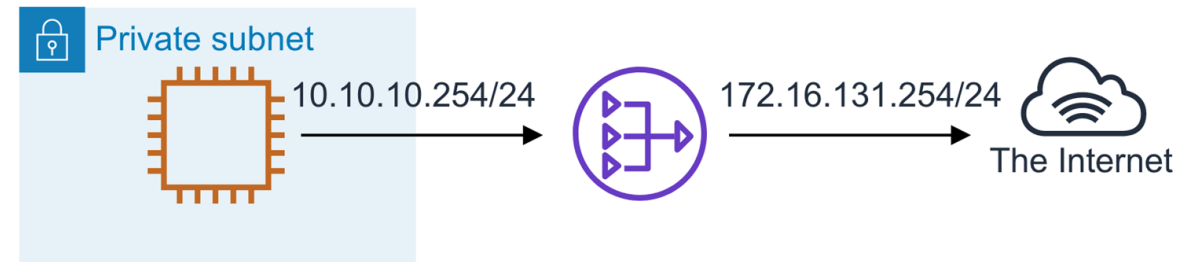
Network Address Translation (NAT)



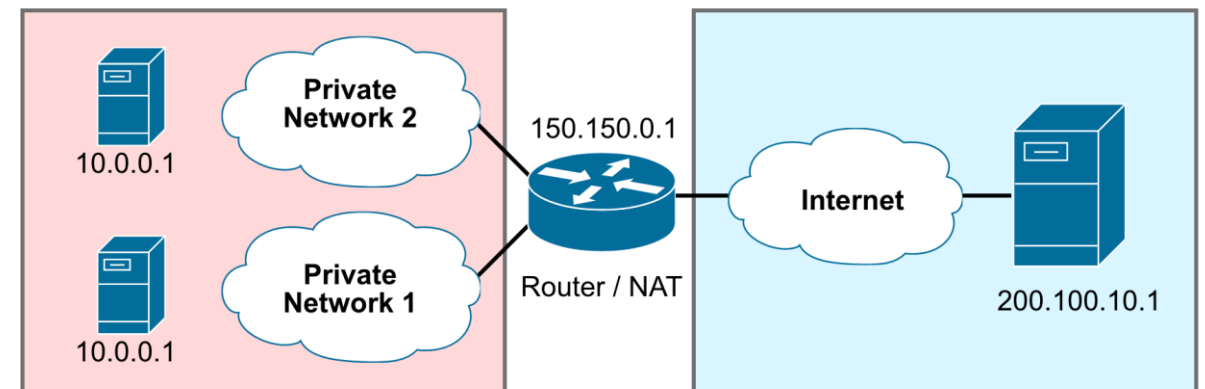
What is Network Address Translation (NAT)?

A method of mapping an IP address space into another by modifying network address information in the IP header of packets while they are in transit across a traffic-routing device

If you have a private network and you need to help gain outbound access to the internet, you would need to use a NAT gateway to re-map the Private IPs



If you have two networks that have conflicting network addresses, you can use a NAT to make the addresses more agreeable



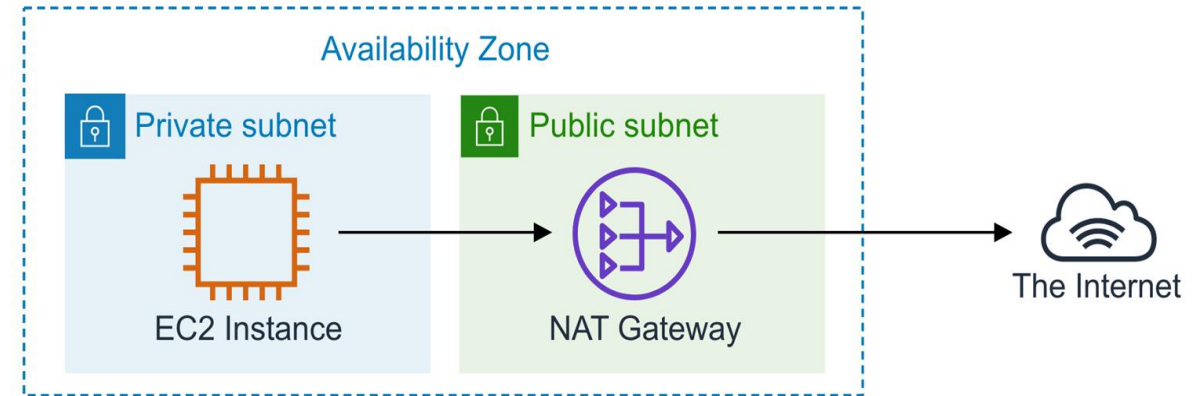
NAT Gateway

NAT Gateway is a **fully managed NAT service** to allow instances in your private subnet to **establish outbound connections**

- A NAT Gateway is redundant within a single subnet
- You need a NAT Gateway per subnet

You pay:

- Per hour per NAT Gateway eg. \$0.045
- Per GB data processed \$0.045
- Eg. 1 Month and 3 NATs = \$98.55



NAT Gateway can get expensive very quickly.

NAT Gateway has two connection modes:

- **Public (Default)**
 - Instances in private subnets can connect to the internet through a public NAT gateway
 - Cannot receive unsolicited inbound connections from the internet
 - You must associate an Elastic IP (EIP) address
- **Private**
 - Instances in private subnets can connect to other VPCs or your on-premises network through a private NAT gateway.
 - You can route traffic from the NAT gateway through a transit gateway or a virtual private gateway.
 - You cannot associate an elastic IP address with a private NAT gateway

DNS64 and NAT64

A NAT gateway supports network address translation **from IPv6 to IPv4**, popularly known as NAT64.

What is DNS64?

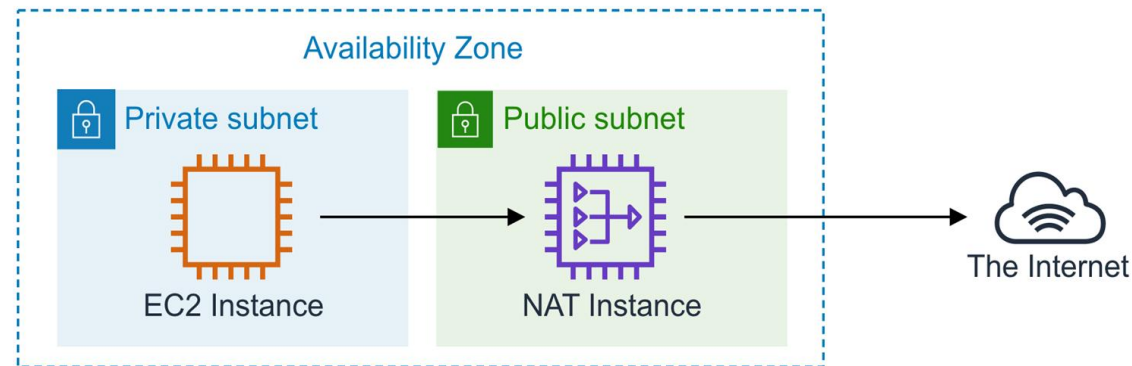
Your IPv6-only workloads running in VPCs can only send and receive IPv6 network packets. Without DNS64, a DNS query for an IPv4-only service will yield an IPv4 destination address in response and your IPv6-only service cannot communicate with it. To bridge this communication gap, you can enable DNS64 for a subnet and it applies to all the AWS resources within that subnet.

What is NAT64?

NAT64 enables your IPv6-only services in Amazon VPCs to communicate with IPv4-only services within the same VPC (in different subnets) or connected VPCs, in your on-premises networks, or over the internet.

NAT Instances

NAT Instances (legacy) is an AWS managed IAM to launch a NAT onto an individual EC2 instances. NAT Instances required the customer to handle scaling



NAT AMI is built on the last version of the Amazon Linux AMI, 2018.03, which reached the end of standard support on December 31, 2020 and end of maintenance support on December 31, 2023



Community AMIs for third-party NAT software exist to launch NAT Instances.



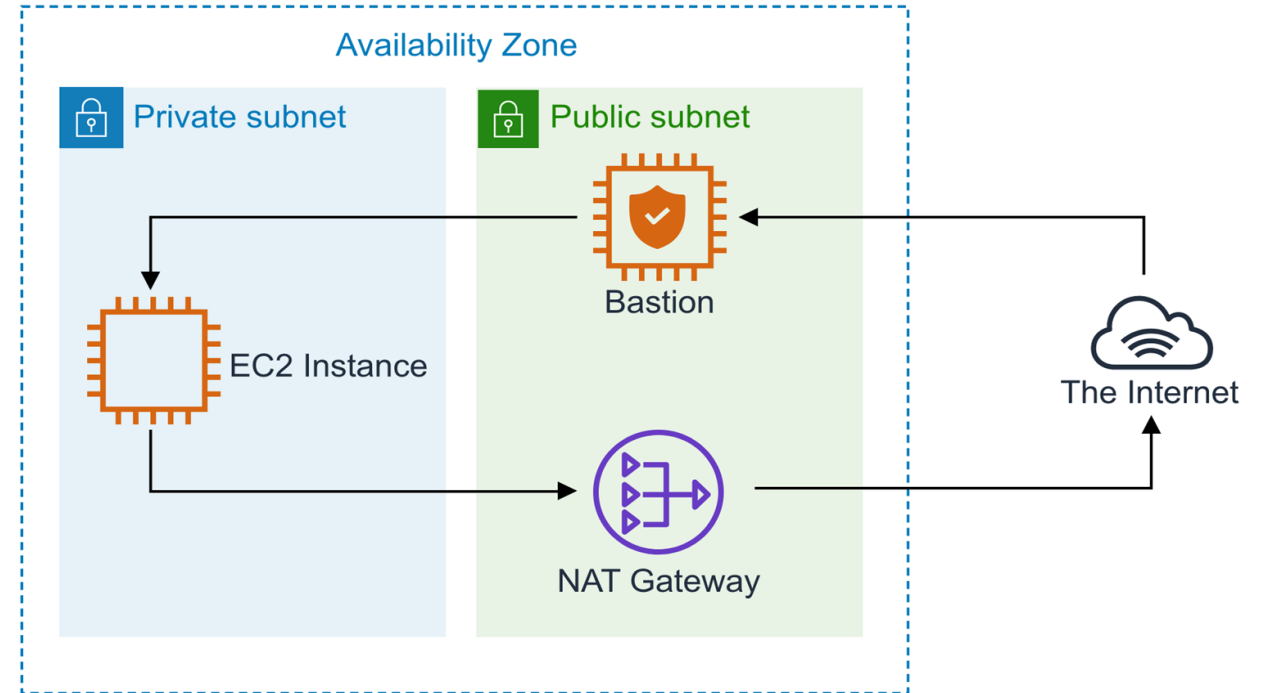
NAT Instance and NAT Gateway CheatSheet

- When creating a NAT instance, you **must disable source and destination checks** on the instance
 - NAT instances **must exist in a public subnet**
 - You must have a **route out** of the private subnet to the NAT instance
 - The size of a NAT instance determines **how much traffic can be handled**
 - **High availability** can be achieved using **Autoscaling Groups**, multiple **subnets in different AZs**, and **automated failover between them using a script**.
-
- NAT Gateways are **redundant inside an Availability Zone** (can survive failure of EC2 instance)
 - You can only have **1 NAT Gateway inside 1 Availability Zone** (cannot span AZs)
 - Starts at 5 Gbps and scales up to 45 Gbps
 - NAT Gateways are the **preferred setup for enterprise systems**.
 - There is no **requirement to patch NAT Gateways**, and there is no **need to disable Source/Destination checks** for the NAT Gateway (unlike NAT Instances)
 - NAT Gateways are **automatically assigned a public IP address**
 - **Route Tables** for the NAT Gateway MUST be updated
 - Resources in multiple AZs sharing a Gateway will **lose internet access if the Gateway** goes down unless you create a **Gateway in each AZ** and configure **route tables** accordingly

Bastion / Jumpbox

Jumpboxes are security-hardened virtual machines that provide **secure access to private subnets**.

- **EC2 Instances** that are security hardened
- Access Methods: Use **SSH** or **RDP (Remote Desktop Protocol)** for remote access
- Known as **jumpboxes** because they allow you to “jump” from one machine to another
- Known as **bastions** because they act as protection against attacks
- Allow access to private subnet resources
- **NAT Gateways** or **NAT Instances** cannot function as Bastions. They are designed to provide outbound access to the internet for resources like software updates

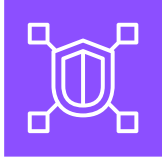


AWS Marketplace offers Community or Third-Party Jumpboxes



System Manager's **Sessions Manager** *can* eliminate the need for Bastions

VPC Lattice



Amazon VPC Lattice is a fully managed application networking service that you use to connect, secure, and monitor the services for your application.

Easily turn your AWS resources into services for a micro-services architecture.

- Can be used in a single VPC
- Can be used across multiple VPCs
- Can be used across accounts and used with multiple accounts
- Performs NAT for IPv4, IPv6, and overlapping networks
- Integrates with IAM
- Weight routing for traffic eg blue/green or canary style deployments
- Uses HTTP/S for service to service communication
- Supports custom domain names
- You can Bring-Your-Own SSL/TLS Certificate

VPC Lattice

Service Network

The logical container for all services which can communicate with each other in the associated VPCs

Listener

The protocol and port the service listens to.

A service can have up-to 2 listeners.

A listener containing routing rules and has a default rule

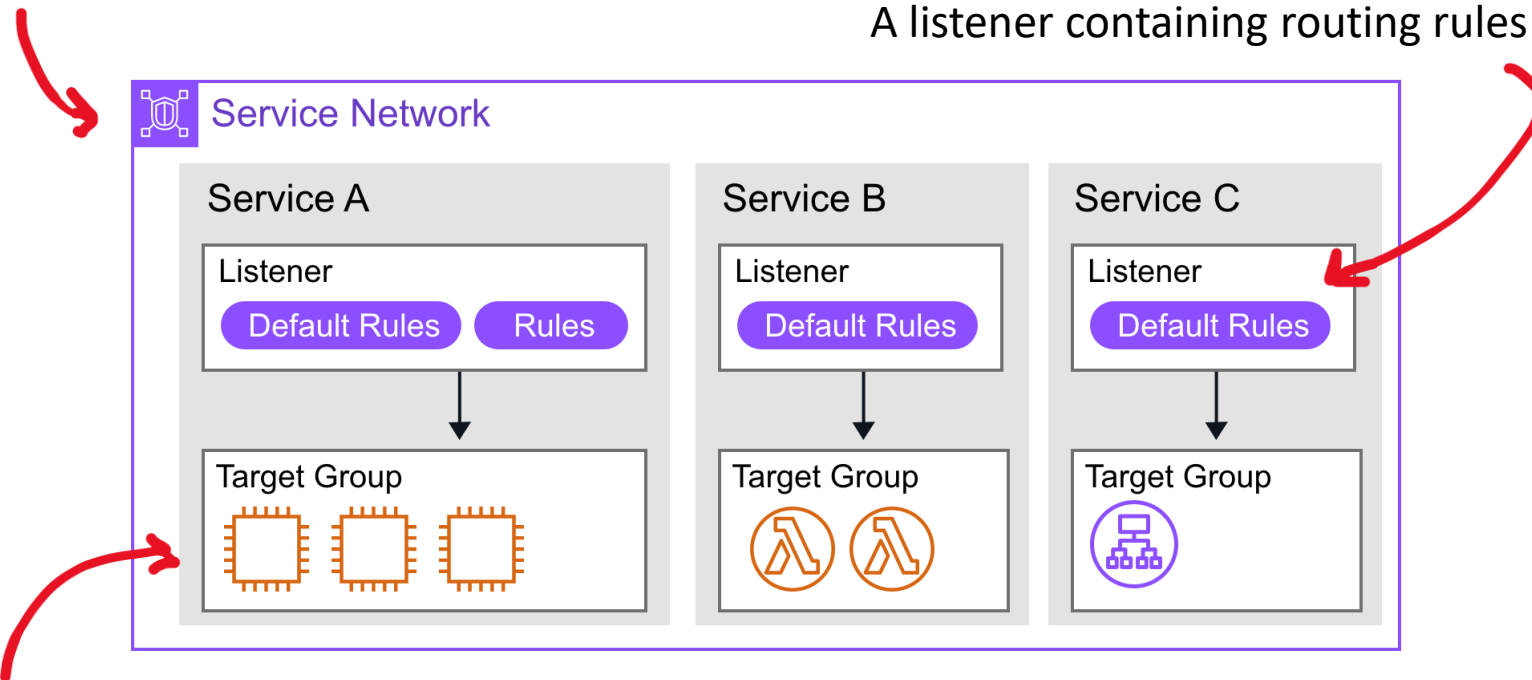
Target Group

A collection of resources of a specific type.

EC2, IP addresses, Lambda Functions, ALBS, K8s pods

Service Directory

A central registry of all VPC Lattice services that you own or are shared with your account through AWS Resource Access Manager (AWS RAM)



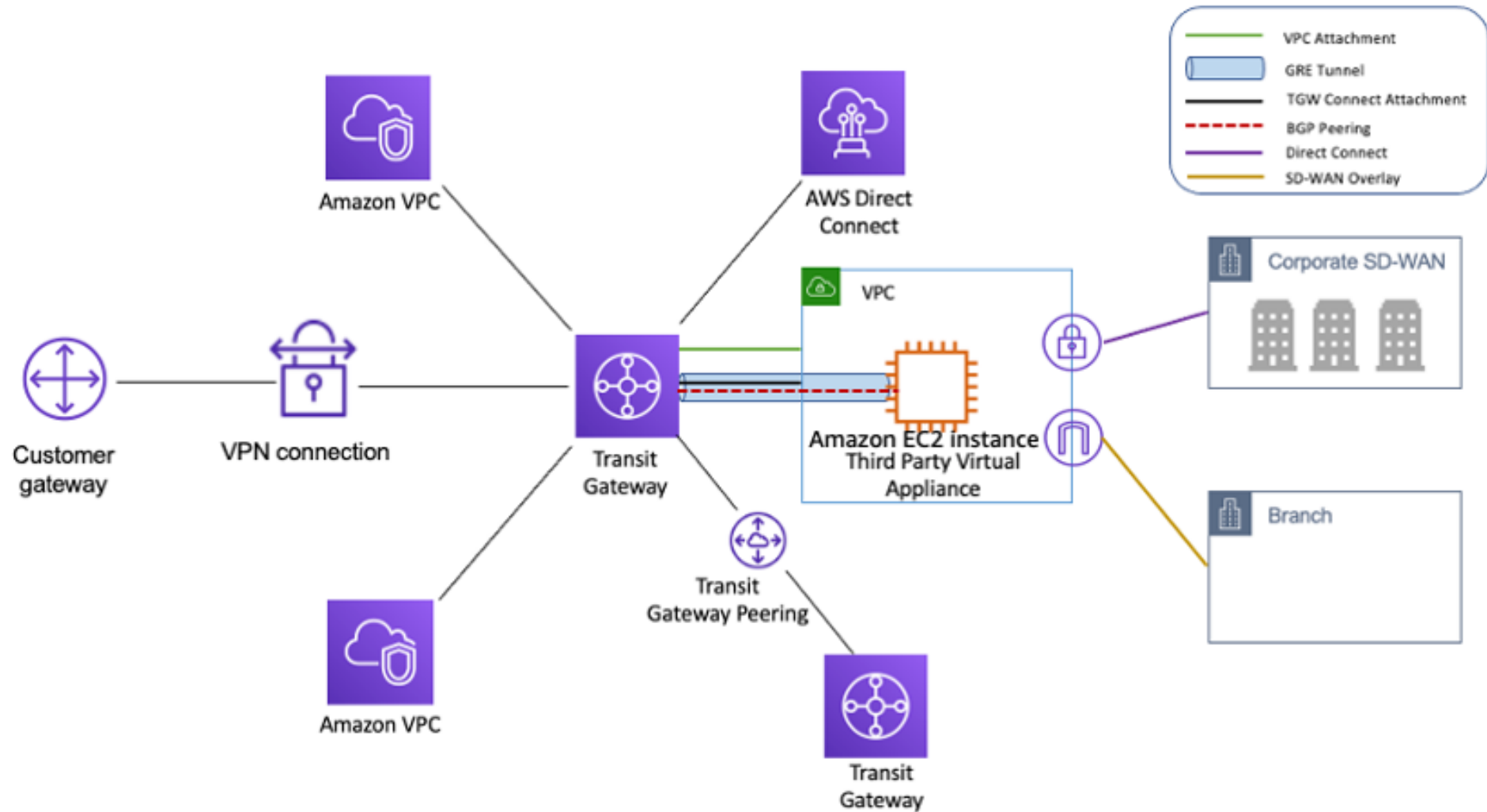
Transit Gateway



A transit gateway is a network transit hub that you can use to interconnect your virtual private clouds (VPCs) and on-premises networks.

- Transit Gateway leverages **AWS Resource Manager (RAM)**
- Operates as a virtual router at the regional level.
- You can attach up to 5000 VPCs to each gateway
 - An ENI will be provisioned in target VPCs to facilitate VPC to Transit Gateway communication
- Each attachment can handle up to 50 Gbits/second of traffic
- You can attach your AWS VPN connections to a Transit Gateway
- You can attach Direct Connect connections to a Transit Gateway
- You can attach third-party virtual appliances via Transit Gateway Attachments
 - can be a source and destination for packets (bi-directional)
- You can peer connection with other Transit Gateways

Transit Gateway



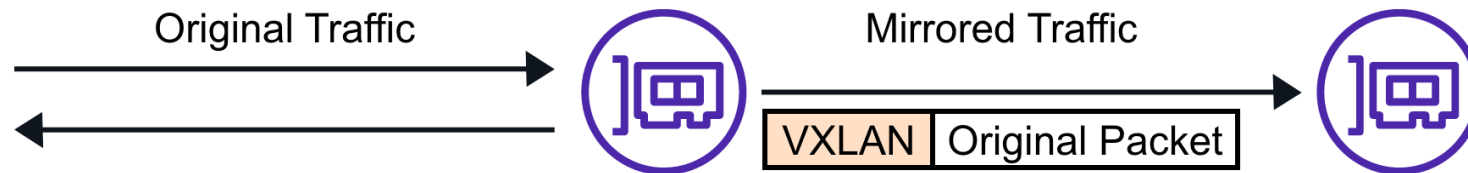
Traffic Mirroring



Traffic Mirroring sends a copy network traffic from a source ENI to target ENI, or UDP-enabled NLB or GWLB



You can use traffic mirroring to send a copy of your traffic to a security monitoring appliance.



- Traffic mirroring will attach a VXLAN header
- Each packet is mirrored once
- Traffic mirroring allows you to apply filter rules
 - Number to prioritize rules (lower is higher priority)
 - Reject or Accept
 - Protocol eg. TCP
 - Source Port Range
 - Destination Port Range
 - Source CIDR block
 - Destination CIDR block

A traffic mirror sessions need to be created to mirror traffic:

- You create a mirror source
- You create a mirror target
- You create a mirror filter

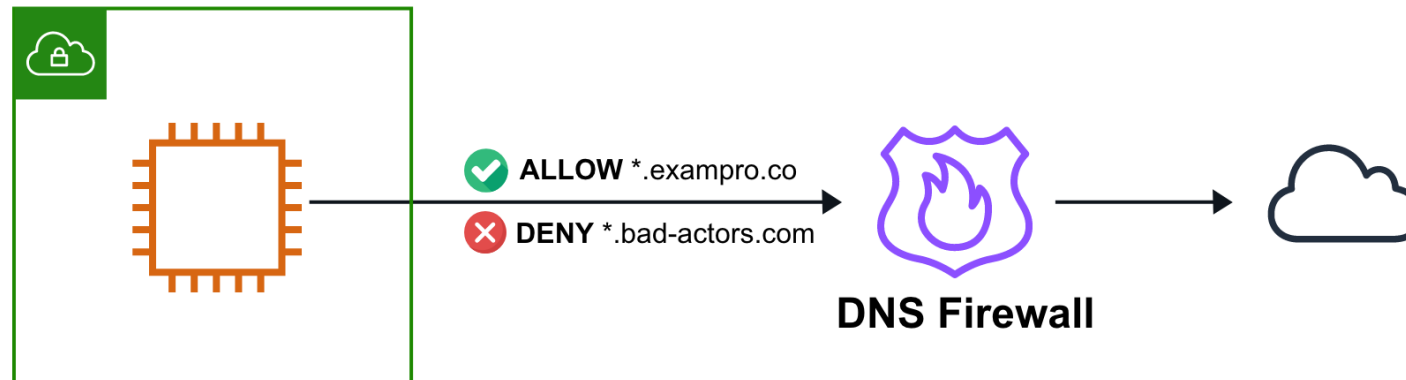
Route 53 Resolver DNS Firewall



Route53 Resolver DNS Firewall is a DNS firewall that **protect against DNS exfiltration of your data.**

What is DNS exfiltration?

DNS data exfiltration is a method used by hackers to steal data from an IT system or network by exploiting the Domain Name System (DNS) protocol by hiding malicious code within DNS packets.



DNS Firewall allows you to monitor and control the domains that your applications can query.

DNS Firewall lets you set Domain Lists:

- Allow rules to only allow trusted domains
- Deny rules deny to prevent domain with known bad-actors

You can centralize your DNS Firewall policies via AWS Firewall Manager

AWS Network Firewall



AWS Network Firewall is a stateful, managed, **network firewall and IDS/IPS for VPCs**

Filter outbound and inbound traffic at the perimeter of your VPC

- Filter IGW traffic
- Filter NAT gateway traffic
- Filter VPN traffic
- Filter Amazon Direct connect traffic



AWS Network Firewall utilizes the open source **Suricata** IDS/IPS software and supports Suricata compatible rules

AWS Network Firewall has multiple use-cases

- Pass traffic through only from known AWS service domains or IP address endpoints, eg Amazon S3.
- Use custom lists of known bad domains to limit the types of domain names that your apps can access.
- Perform deep packet inspection on traffic entering or leaving your VPC.
- Use stateful protocol detection to filter protocols like HTTPS, or an independent port is used.

AWS Verified Access



AWS Verified Access provides **secure access to your AWS resources** **without the need of a VPN** by **using trusted connected providers**